QUẢN LÝ HỒ SƠ SINH VIÊN

Nhóm 8

Giảng viên: Trần Anh Quân

THÀNH VIÊN

Nguyễn Hồ Diễm My

Nguyễn Hồ Hùng Phương

Hồ Thị Ngọc Mỹ

Huỳnh Gia Huy

Huỳnh Thái Vỹ

GIAO TÁC (TRANSACTION)

Khái niệm

- Giao tác là một đơn vị xử lý logic gồm nhiều thao tác đọc/ghi dữ liệu.
- Các thao tác trong giao tác được xử lý như một thể thống nhất.
- Mục tiêu: đảm bảo tính nhất quán và an toàn của CSDL.



GIAO TÁC (TRANSACTION)



Tính chất ACID

Tính nguyên tố (Atomicity)

Tính nhất quán (Consistency)

Tính nhất quán (Consistency)

Tính bền vững (Durability)

Các trạng thái của giao tác

Một giao tác trong quá trình thực hiện sẽ lần lượt trải qua các trạng thái sau:

- **Active:** trạng thái ngay sau khi giao tác bắt đầu thực hiện các hành động đọc/ghi.
- **Partially Committed:** trạng thái sau khi lệnh cuối cùng của giao tác được thực hiện.
- **Committed:** trạng thái sau khi giao tác hoàn thành thành công.
- **Failed:** trạng thái xảy ra khi phát hiện rằng không thể tiếp tục thực hiện các hành động của giao tác được nữa.
- **Aborted:** trạng thái sau khi giao tác đã bị cuộn lại (rollback) và CSDL đã được phục hồi về trạng thái trước khi giao tác bắt đầu.
- **Terminated:** trạng thái chấm dứt giao tác (sau khi Committed hoặc Aborted).

CÚ PHÁP THỰC HIỆN GIAO TÁC TRONG SQL

Thực hiện Transaction

```
BEGIN TRANSACTION

    -- Các thao tác xử lý dữ liệu

    COMMIT TRANSACTION
```

Nếu xảy ra lỗi:

```
ROLLBACK TRANSACTION
```

THỰC THI

Thực hiện giao tác THÊM sinh viên

QUẢN LÝ HỒ SƠ SINH VIÊN

Mã sinh viên:

Họ tên:

CCCD:

Ngày sinh: 1/ 1/2005

Giới tính:

Số điện thoại:

Email:

Địa chỉ:

Dân tộc:

Trạng thái học tập:

Lớp:

Thêm | Sửa | Xóa | Làm mới | Số SV trong lớp | Lịch sử xóa

Tìm kiếm: Tìm

	Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
▶	SV01	Nguyễn Văn An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Đang học	L01
	SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
	SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.co...	Vĩnh Long	Khơ me	Đang học	L02
	SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
	SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
	SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
	SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
	SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07



Bước 1: Nhập đầy đủ thông tin sinh viên trên giao diện.

Bước 2: Nhấn nút “Thêm” để thực hiện thao tác.



Bước 3: GUI lấy dữ liệu và tạo đối tượng SinhVienDTO, sau đó truyền xuống tầng BUS.

Bước 4: BUS kiểm tra tính hợp lệ của dữ liệu và mã sinh viên có bị trùng hay không.

Bước 5 : DAL kết nối CSDL và gọi Stored Procedure sp_ThemSinhVien để thêm dữ liệu.

```sql
-- Tạo Procedure thêm Sinh Viên
create procedure sp_ThemSinhVien
    @MaSV varchar(10),
    @HoTen nvarchar(50),
    @CCCD varchar(20),
    @NgaySinh date,
    @GioiTinh nvarchar(10),
    @SDT varchar(20),
    @Email varchar(50),
    @DiaChi nvarchar(100),
    @DanToc nvarchar(20),
    @TrangThaiHocTap nvarchar(50),
    @MaLop varchar(10)
as
begin
    insert into SinhVien
    (MaSV,HoTen,CCCD,
    NgaySinh,GioiTinh,SDT,
    Email,DiaChi,DanToc,
    TrangThaiHocTap,MaLop)
    values
    (@MaSV,@HoTen,@CCCD,@NgaySinh,@GioiTinh,@SDT,
    @Email,@DiaChi,@DanToc,@TrangThaiHocTap,@MaLop);
end;
go
```

- **Tầng DTO:** Khai báo lớp SinhVienDTO để đóng gói thông tin sinh viên và truyền dữ liệu giữa các tầng.
- **Tầng BUS:** Tiếp nhận SinhVienDTO, kiểm tra dữ liệu hợp lệ và kiểm tra mã sinh viên đã tồn tại hay chưa.
- **Tầng DAL:** Kết nối CSDL và gọi Stored Procedure dbo.sp_ThemSinhVien để thực hiện thêm dữ liệu.
- **Tầng SQL:** Stored Procedure nhận thông tin sinh viên và thực hiện câu lệnh INSERT vào bảng SinhVien.

VIEW

Khái niệm:

- View là một đối tượng CSDL ảo, được tạo từ một câu lệnh ‘ SELECT ’ đã lưu trữ sẵn trong hệ quản trị cơ sở dữ liệu.
- View chủ yếu lưu định nghĩa truy vấn, không lưu dữ liệu vật lý riêng.
- Khi được gọi, SQL Server thực thi lại câu lệnh SELECT để lấy dữ liệu hiện hành.
- View là “cửa sổ” giúp người dùng xem dữ liệu từ một hoặc nhiều bảng theo góc nhìn được định sẵn, mà không cần quan tâm cấu trúc bên trong.

VIEW

Đặc điểm:

- **Không lưu trữ dữ liệu thực :**

+ View chủ yếu lưu định nghĩa truy vấn, không lưu bản sao dữ liệu.
+ Dữ liệu hiển thị luôn được lấy trực tiếp và cập nhật theo thời gian thực từ bảng gốc.

- **Bảo mật và kiểm soát truy cập:**

+ View có thể giới hạn người dùng chỉ được xem một số dòng hoặc cột.
+ Đây là cơ chế bảo mật ở mức logic giúp bảo vệ dữ liệu trong bảng gốc.

- **Đơn giản hóa truy vấn phức tạp :**

+ View đóng gói các truy vấn có JOIN, GROUP BY,...lại thành một đối tượng duy nhất.
+ Người dùng chỉ cần gọi View thay vì lặp lại câu lệnh phức tạp nhiều lần.

‘ SELECT * FROM TenView ’

VIEW

Đặc điểm:

- **Tính độc lập dữ liệu :**

+ View tạo lớp trung gian giữa ứng dụng và cấu trúc CSDL.
+ Có thể điều chỉnh View khi cấu trúc bảng gốc thay đổi.

- **Tính động (Dynamic):**

+ View không lưu bản sao dữ liệu.
+ Khi dữ liệu bảng gốc thêm, sửa, xóa, View sẽ phản ánh thay đổi khi được truy vấn.

- **Giới hạn khi cập nhật dữ liệu :**

+ Không phải View nào cũng hỗ trợ INSERT, UPDATE, DELETE.
+ Khả năng cập nhật phụ thuộc vào cấu trúc của View.

VIEW

Cú pháp cơ bản:

```
CREATE VIEW TenView
AS
SELECT cot1, cot2, ...
FROM TenBang
WHERE dieu_kien ;
SELECT * FROM TenView;
```

- **CREATE VIEW :** Lệnh dùng để khởi tạo View.
- **TenView :** Tên mà mình muốn đặt cho View
- **AS :** Từ khóa phân cách giữa tên view và câu lệnh truy vấn.
- **SELECT cot1, cot2,...:** Truy vấn lấy dữ liệu từ 1 bảng hay nhiều bảng.
- **FROM TenBang:** lấy từ bảng đó tên gì.
- **WHERE:** dieu_kien : Điều kiện để lấy dữ liệu.
- **SELECT * FROM TenView; :** Truy vấn, hiển thị toàn bộ dữ liệu View vừa tạo.

VIEW

Ưu điểm và Nhược điểm:

Ưu điểm:

- Đơn giản hóa việc viết truy vấn.
- Tăng cường bảo mật dữ liệu.
- Hỗ trợ tính độc lập logic giữa ứng dụng và CSDL.
- Cho phép tái sử dụng logic truy vấn.
- Indexed View có thể hỗ trợ tối ưu hiệu năng.

Nhược điểm:

- View phức tạp có thể ảnh hưởng hiệu năng.
- Không phải View nào cũng hỗ trợ INSERT/UPDATE/DELETE.
- Indexed View có chi phí bảo trì cao.
- Lạm dụng View lồng nhau khiến việc tối ưu truy vấn khó hơn.

VIEW

```sql
-- Tạo View hiển thị danh sách sinh viên
CREATE VIEW dbo.vw_SinhVienDayDu
AS
SELECT
    sv.MaSV,
    sv.HoTen,
    sv.CCCD,
    sv.NgaySinh,
    sv.GioiTinh,
    sv.SDT,
    sv.Email,
    sv.DiaChi,
    sv.DanToc,
    sv.TrangThaiHocTap,
    sv.MaLop,
    l.TenLop,
    l.NienKhoa,
    n.MaNganh,
    n.TenNganh,
    k.MaKhoa,
    k.TenKhoa
FROM dbo.SinhVien sv
JOIN dbo.Lop l
    ON sv.MaLop = l.MaLop
JOIN dbo.Nganh n
    ON l.MaNganh = n.MaNganh
JOIN dbo.Khoa k
    ON n.MaKhoa = k.MaKhoa;
GO
```

- Tạo View dbo.vw_SinhVienDayDu bằng cách lấy dữ liệu từ bảng SinhVien và kết nối với các bảng Lop, Nganh, Khoa thông qua các khóa liên kết.
- View sử dụng câu lệnh SELECT như MaSV, HoTen, CCCD,... để lấy thống tin sinh viên.
- View được sử dụng dùng để hiển thị danh sách sinh viên đầy đủ trên giao diện

VIEW

- **Mô tả:** Tạo các View bằng cách kết hợp dữ liệu từ nhiều bảng, nhằm hiển thị thông tin sinh viên, lớp, ngành, người thân và chương trình đào tạo, đồng thời thống kê số lượng sinh viên theo lớp và ngành.
- **Ví dụ:** Thực thi tạo View hiển thị danh sách sinh viên

VIEW

Thực thi: Tạo View hiển thị danh sách sinh viên

B1: GUI mở giao diện quản lý sinh viên, hệ thống tự động tải danh sách sinh viên và hiển thị lên bảng DataGridView. GUI gửi yêu cầu lấy dữ liệu đến tầng BUS.

B2: BUS Nhận yêu cầu từ GUI, gọi GetAll() lấy danh sách sinh viên, sau đó trả kết quả về GUI.

B3: DAL kết nối SQL Server, truy vấn lấy DSSV từ View vw_SinhVienDayDu rồi sắp xếp theo MaSV và trả dữ liệu về BUS.

B4: SQL xây dựng View vw_SinhVienDayDu bằng cách kết nối các bảng SinhVien, Lop, Nganh và Khoa lại với nhau.Sau đó view sẽ cung cấp thông tin đầy đủ của sinh viên.

	Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
▶	SV01	Nguyễn Văn An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Đang học	L01
	SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
	SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
	SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
	SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
	SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
	SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
	SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
	SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
	SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09

VIEW

Giao diện view hiển thị danh sách sinh viên

QUẢN LÝ HỒ SƠ SINH VIÊN

Mã sinh viên:
Họ tên:
CCCD:
Ngày sinh: 1/ 1/2005
Giới tính:
Số điện thoại:

Email:
Địa chỉ:
Dân tộc:
Trạng thái học tập:
Lớp:

Thêm **Sửa** **Xóa** **Làm mới** **Số SV trong lớp** **Lịch sử xóa**

Tìm kiếm: **Tìm**

	Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
▶	SV01	Nguyễn Văn An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Đang học	L01
	SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
	SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
	SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
	SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
	SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
	SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
	SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
	SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
	SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09

STORED PROCEDURE

Khái niệm:

- Stored Procedure là tập hợp 1 hoặc nhiều câu lệnh T-SQL thành 1 nhóm đơn vị xử lí logic, đc lưu trên Database Server
- Khi câu lệnh gọi chạy stored procedure lầu đầu thì SQL Server chạy, lưu trữ vào bộ nhớ đệm (plan cache), những lần tiếp theo sẽ sử dụng lại plan cache => tốc độ xử lí sẽ tối ưu.
- Tiện lợi cho người quản trị database, giúp tạo ra những nhóm câu lệnh và gửi đến 1 bộ phận khác mà họ kh cần quan tâm đến nd ở trong stored procedure, chỉ quan tâm tham số đầu vào/ra.

STORED PROCEDURE

Đặc điểm:



- Lưu trữ trực tiếp trong CSDL, có thể gọi lại nhiều lần khi cần.
- Nhận tham số đầu vào (input) truyền dữ liệu, trả về tham số đầu ra (output) đưa kết quả.
- Chứa nhiều câu lệnh SQL trong cùng một thủ tục.
    - Có thể sử dụng các câu lệnh điều kiện if-else, vòng lặp,các câu lệnh xử lí khác.
    - Thực hiện các thao tác: insert, update,delete,select
    - Có thể gọi lại và tái sử dụng để giảm viết lại SQL

STORED PROCEDURE

Cú pháp cơ bản

```
CREATE PROCEDURE sp_TenProcedure
    @ThamSo KieuDuLieu
AS
BEGIN
SET NOCOUNT ON;
END;
EXEC sp_TenProcedure;
```

- CREATE PROCEDURE : Lệnh khởi tạo stored procedure.
- sp_TenProcedure : Tên của stored procedure.
- @ThamSo KieuDuLieu : Tham số và kiểu dữ liệu.
- AS....BEGIN : các câu lệnh SQL xử lí logic trong procedure.
- SET NOCOUNT ON; : Không hiển thị thông báo số dòng bị thay đổi.
- EXEC sp_TenProcedure; : Thực thi stored procedure.

STORED PROCEDURE

Ưu điểm và Nhược điểm:

- Ưu điểm:

+ Tái sử dụng nhiều lần, giảm viết lại các câu lệnh SQL.

+ Xử lí nhiều câu lệnh SQL trong cùng 1 thủ tục.

+ Truyền tham số giúp stored procedure linh hoạt hơn.

+ Tăng tính bảo mật do người dùng đc cấp quyền thực thi stored procedure thay vì truy cập trực tiếp vào bảng.

+ Giúp chương trình dễ tổ chức, quản lí các nghiệp vụ về CSDL.

STORED PROCEDURE

Ưu điểm và Nhược điểm:

- Nhược điểm:

+ Nếu stored procedure quá dài , chứa nhiều các lệnh xử lí phức tạp thì khi lỗi khó bảo trì.

+ Nó phụ thuộc vào hệ quản trị CSDL.

+ Có quá nhiều stored procedure thì quản lí trở nên khó và phức tạp hơn.

+ Khi thay đổi CSDL hay cấu trúc thì stored procedure nào liên quan thì phải thay đổi theo.

STORED PROCEDURE

```
-- Tạo Procedure thêm Sinh Viên
create procedure sp_ThemSinhVien
    @MaSV varchar(10),
    @HoTen nvarchar(50),
    @CCCD varchar(20),
    @NgaySinh date,
    @GioiTinh nvarchar(10),
    @SDT varchar(20),
    @Email varchar(50),
    @DiaChi nvarchar(100),
    @DanToc nvarchar(20),
    @TrangThaiHocTap nvarchar(50),
    @MaLop varchar(10)
as
begin
    insert into SinhVien
    (MaSV,HoTen,CCCD,
    NgaySinh,GioiTinh,SDT,
    Email,DiaChi,DanToc,
    TrangThaiHocTap,MaLop)
    values
    (@MaSV,@HoTen,@CCCD,@NgaySinh,@GioiTinh,@SDT,
    @Email,@DiaChi,@DanToc,@TrangThaiHocTap,@MaLop);
end;
go
```

- **GUI:** Người dùng nhập thông tin lớp trên giao diện MaSV, HoTen,..chọn Thêm Lớp.
- **DTO:** gom các thông tin sinh viên thành đối tượng SinhVienDTO
- **BUS:** Nhận SinhVienDTO, ktra dữ liệu hợp lê kh
- **DAL:** nhận DTO từ BUS sau đó truyền các thuộc tính của DTO vào tham số của sp_ThemSinhVien
- **SQL**: Thực thi thủ tục và thêm sinh viên vào Bảng

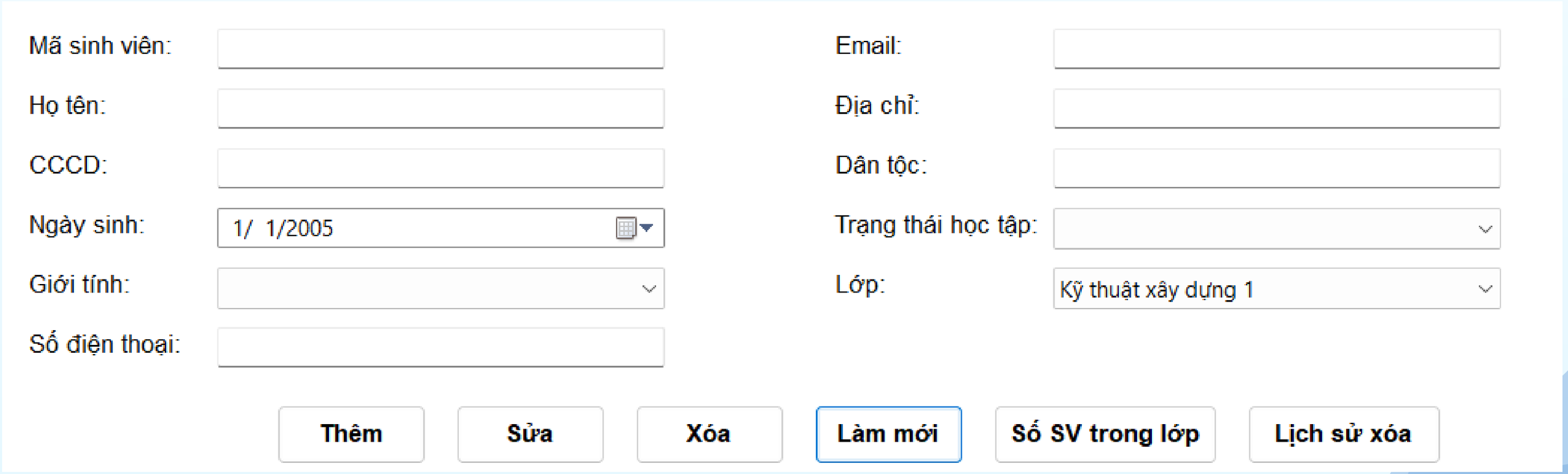
STORED PROCEDURE

- **Mô tả:** Stored Procedure dùng để thực hiện các thao tác thêm, tìm kiếm, sửa, xóa và cập nhật dữ liệu trên các bảng Khoa, Ngành, CTDT, Lớp, Sinh Viên, Người Thân.
- **Ví dụ:** Thực thi Thêm Sinh Viên

STORED PROCEDURE

Thực thi: Tạo SP Thêm sinh viên

Giao diện ban đầu chưa thêm sinh viên



STORED PROCEDURE

Giao diện ban đầu chưa thêm sinh viên

QUẢN LÝ HỒ SƠ SINH VIÊN

Mã sinh viên:
Họ tên:
CCCD:
Ngày sinh: 1/ 1/2005
Giới tính:
Số điện thoại:

Email:
Địa chỉ:
Dân tộc:
Trạng thái học tập:
Lớp: Kỹ thuật xây dựng 1

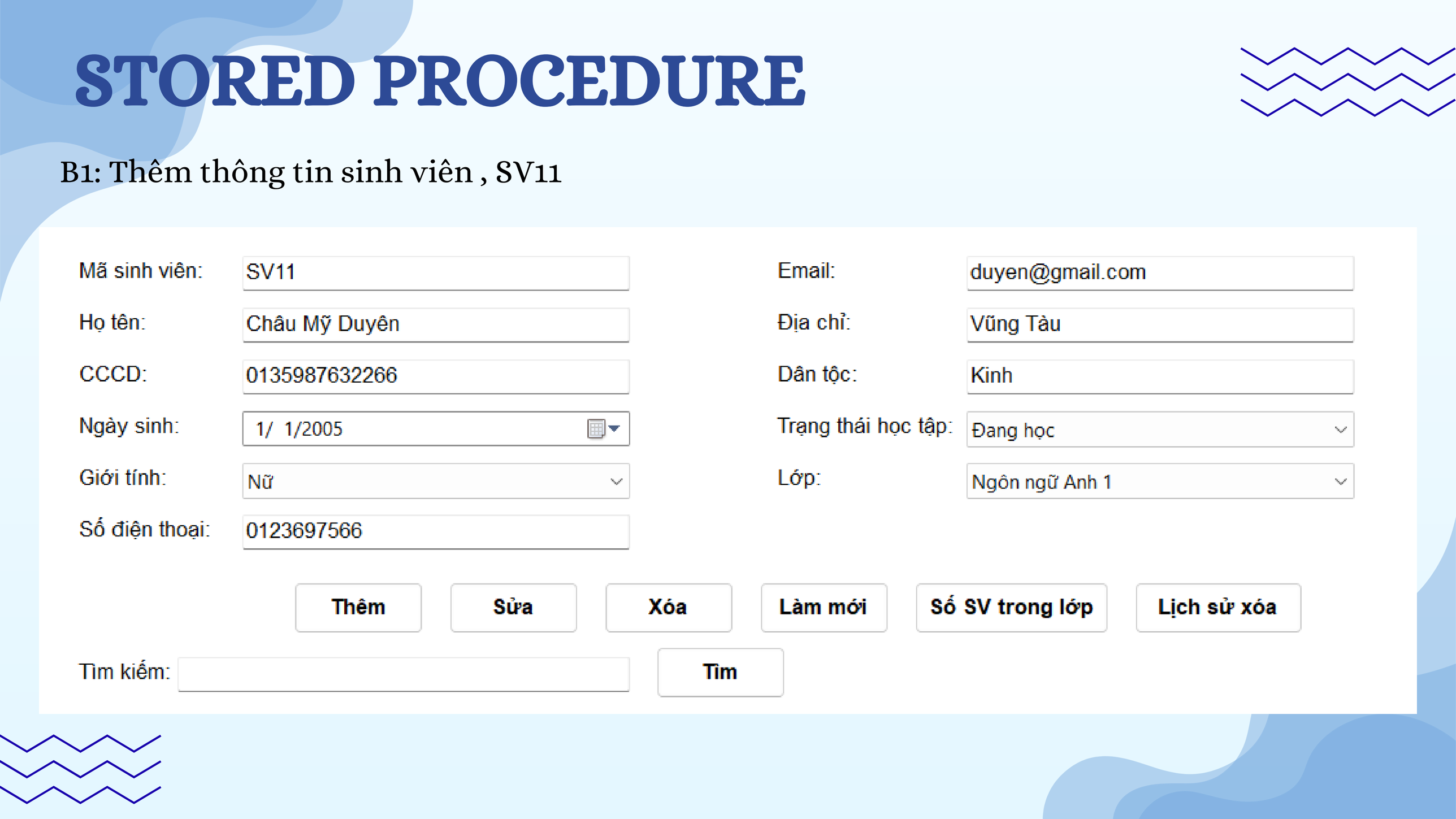
Thêm | Sửa | Xóa | Làm mới | Số SV trong lớp | Lịch sử xóa

Tìm kiếm: Tìm

Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Nguyễn Văn An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Đang học	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09

STORED PROCEDURE

B1: Thêm thông tin sinh viên , SV11



STORED PROCEDURE

B1: Thêm thông tin sinh viên , SV11

QUẢN LÝ HỒ SƠ SINH VIÊN

Mã sinh viên: SV11

Họ tên: Châu Mỹ Duyên

CCCD: 0135987632266

Ngày sinh: 1/18/2005

Giới tính: Nữ

Số điện thoại: 0123697566

Email: duyen@gmail.com

Địa chỉ: Vũng Tàu

Dân tộc: Kinh

Trạng thái học tập: Đang học

Lớp: Ngôn ngữ Anh 1

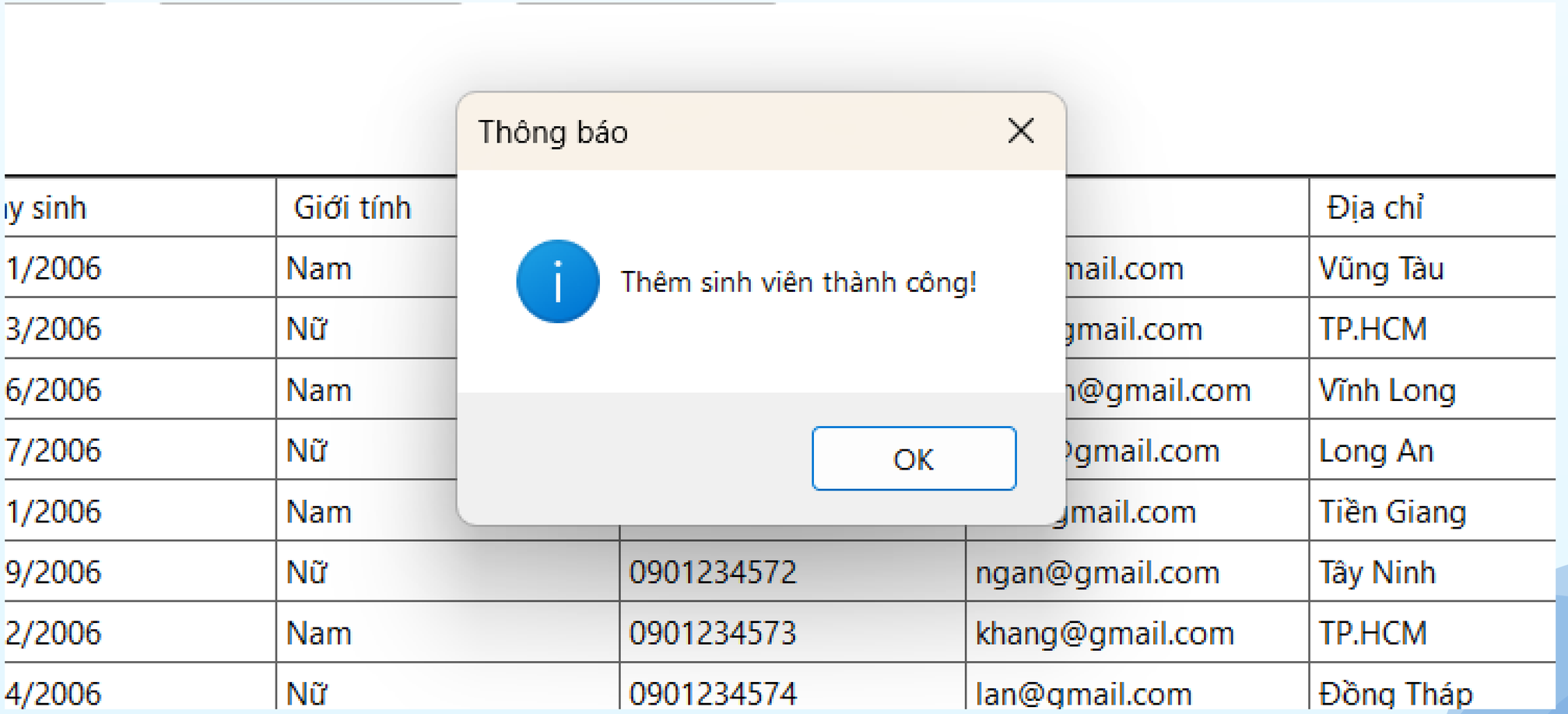
Thêm | Sửa | Xóa | Làm mới | Số SV trong lớp | Lịch sử xóa

Tìm kiếm: Tìm

Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Nguyễn Văn An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Đang học	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09

STORED PROCEDURE

B2: Thêm sinh viên thành công



ıy sinh	Giới tính			Địa chỉ
1/2006	Nam		mail.com	Vũng Tàu
3/2006	Nữ		gmail.com	TP.HCM
6/2006	Nam		n@gmail.com	Vĩnh Long
7/2006	Nữ		gmail.com	Long An
1/2006	Nam		gmail.com	Tiền Giang
9/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh
2/2006	Nam	0901234573	khang@gmail.com	TP.HCM
4/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp

STORED PROCEDURE

B2: Thêm sinh viên thành công

QUẢN LÝ HỒ SƠ SINH VIÊN

Mã sinh viên: SV11
Họ tên: Châu Mỹ Duyên
CCCD: 0135987632266
Ngày sinh: 1/18/2005
Giới tính: Nữ
Số điện thoại: 0123697566

Email: duyen@gmail.com
Địa chỉ: Vũng Tàu
Dân tộc: Kinh
Trạng thái học tập: Đang học
Lớp: Ngôn ngữ Anh 1

Thêm | Sửa | Xóa | Làm mới | Số SV trong lớp | Lịch sử xóa

Tìm kiếm: Tìm

Thông báo

Thêm sinh viên thành công!

OK

	Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính			Địa chỉ	Dân tộc	Trạng thái	Mã lớp
▶	SV01	Nguyễn Văn An	079123456789	28/01/2006	Nam		mail.com	Vũng Tàu	Kinh	Đang học	L01
	SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ		gmail.com	TP.HCM	Kinh	Đang học	L01
	SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam		n@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
	SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ		gmail.com	Long An	Hoa	Đang học	L03
	SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam		gmail.com	Tiền Giang	Kinh	Đang học	L04
	SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
	SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
	SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
	SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
	SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09

STORED PROCEDURE

B3: Đã thêm sinh viên và cập nhật xong

Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Nguyễn Văn An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Đang học	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09
SV11	Châu Mỹ Duyên	0135987632266	18/01/2005	Nữ	0123697566	duyen@gmail.com	Vũng Tàu	Kinh	Đang học	L04

STORED PROCEDURE

B3: Đã thêm sinh viên và cập nhật xong

QUẢN LÝ HỒ SƠ SINH VIÊN

Mã sinh viên: SV11

Họ tên: Châu Mỹ Duyên

CCCD: 0135987632266

Ngày sinh: 1/ 1/2005

Giới tính: Nữ

Số điện thoại: 0123697566

Email: duyen@gmail.com

Địa chỉ: Vũng Tàu

Dân tộc: Kinh

Trạng thái học tập: Đang học

Lớp: Ngôn ngữ Anh 1

Thêm | Sửa | Xóa | Làm mới | Số SV trong lớp | Lịch sử xóa

Tìm kiếm: Tìm

Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Nguyễn Văn An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Đang học	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09
SV11	Châu Mỹ Duyên	0135987632266	18/01/2005	Nữ	0123697566	duyen@gmail.com	Vũng Tàu	Kinh	Đang học	L04

FUNCTION

Function (hàm) trong SQL: một đối tượng quan trọng thuộc hệ quản trị cơ sở dữ liệu, thực hiện các phép tính toán hoặc xử lý dữ liệu và trả về một giá trị duy nhất hoặc một tập kết quả (dạng bảng).
Khác với Stored Procedure, Function luôn bắt buộc phải có giá trị trả về thông qua câu lệnh RETURN.

ĐẶC ĐIỂM NỔI BẬT

Tính Tái Sử Dụng Cao

- Được gọi lại nhiều lần trong các mệnh đề khác nhau: SELECT, WHERE, JOIN, GROUP BY.

Xử Lý Linh Hoạt Với Tham Số

- Hỗ trợ nhận tham số đầu vào để xử lý theo yêu cầu cụ thể.

Gọn gàng và dễ bảo trì:

- Giảm thiểu việc lặp lại mã nguồn, giúp chương trình gọn gàng và dễ bảo trì hơn.

An Toàn và Ổn Định Dữ Liệu

- *Không được dùng* để thực hiện các thao tác làm thay đổi trạng thái dữ liệu trong bảng.
- Chặn các thao tác ghi các lệnh như INSERT, UPDATE, DELETE bên trong hàm, giúp bảo vệ toàn vẹn dữ liệu khi truy vấn.

Lưu ý quan trọng: Function có thể lồng vào biểu thức (điều kiện lọc, tính toán cột) mà Stored Procedure không làm được. Phù hợp cho logic đơn giản và tính toán thay vì nghiệp vụ phức tạp.

FUNCTION

CÚ PHÁP TẠO FUNCTION TRONG SQL

Tạo Function

```
CREATE FUNCTION fn_Tong2So
(
  @a INT,
  @b INT
)
RETURNS INT
AS
BEGIN
  RETURN (@a + @b)
END
```

FUNCTION

ƯU ĐIỂM VÀ NHƯỢC ĐIỂM

Ưu Điểm

Tái Sử Dụng Cao

- Đóng gói khối logic tính toán và gọi lại ở nhiều nơi (SQL, DAL,...) mà không cần viết lại mã nguồn.

Trừu Tượng Hóa

- Đơn giản hóa các câu lệnh truy vấn phức tạp, giữ mã nguồn ở tầng DAL gọn gàng và dễ bảo trì hơn.

Hiệu Suất Tốt

- Tối ưu hóa thực thi khá tốt đối với các scalar function thực hiện các phép tính đơn giá trị.

Nhược điểm

Hạn Chế Thao Tác

- Không được phép thực hiện các lệnh thay đổi trạng thái CSDL như INSERT, UPDATE, DELETE trực tiếp bên trong.

Ảnh Hưởng Hiệu Suất

- Hàm trả về bảng phức tạp (Inline/Multi-statement) có thể làm chậm execution plan khi xử lý dữ liệu lớn.

Khó Gỡ Lỗi

- Việc thực hiện debug lỗi bên trong các Function phức tạp khó khăn hơn so với Stored Procedure.

THỰC THI

TẠO FUNCTION ĐẾM SỐ SINH VIÊN TRONG 1 LỚP

- **Bước 1: Chọn một dòng sinh viên bất kỳ thuộc lớp cần xem (ví dụ: sinh viên có mã lớp L01) trên bảng dữ liệu.**
- **Bước 2: Nhấn vào nút “Số SV trong lớp”.**

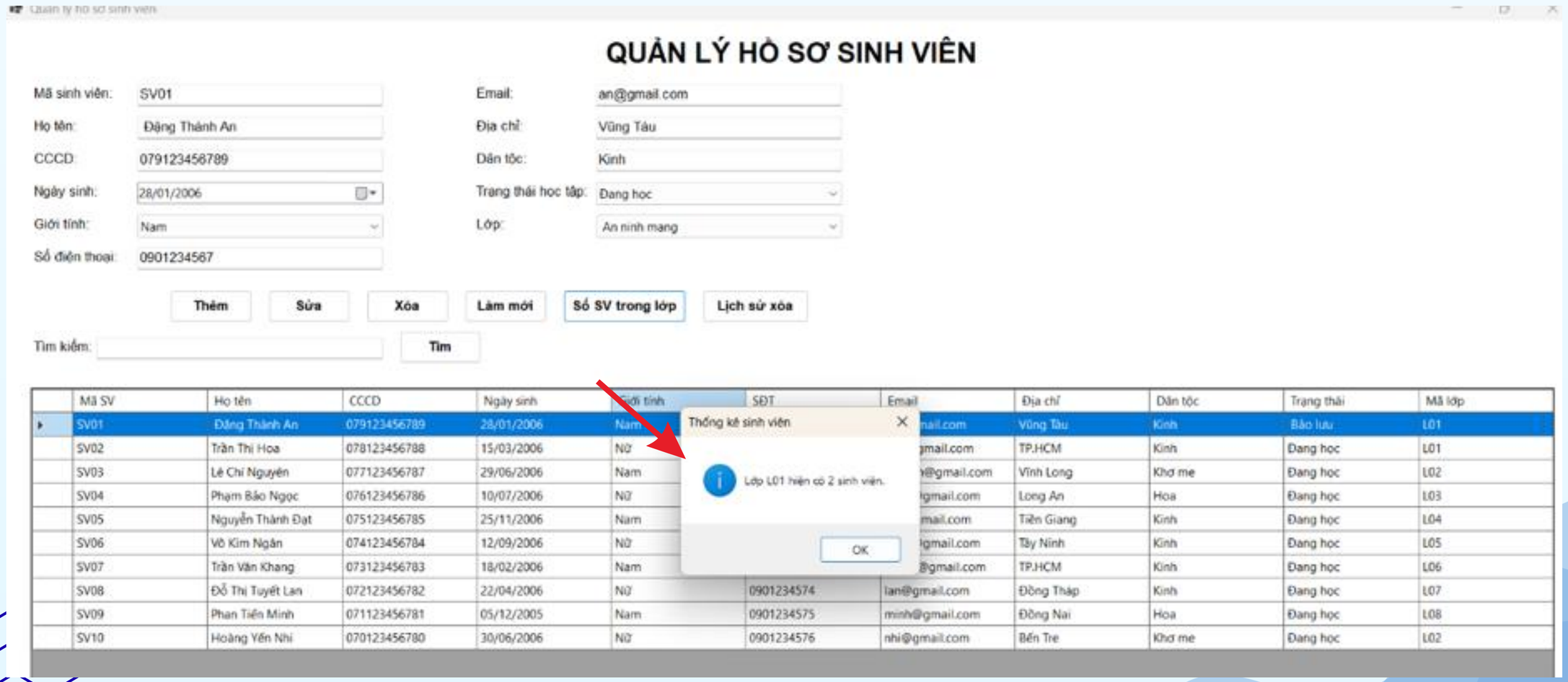


Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Đặng Thành An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Bảo lưu	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L02

THỰC THI

TẠO FUNCTION ĐẾM SỐ SINH VIÊN TRONG 1 LỚP

- **Bước 3: Số sinh viên có trong lớp sẽ được thể hiện ở bảng. (Lớp L01 hiện có 2 sinh viên)**



THỰC THI

TẠO FUNCTION ĐẾM SỐ SINH VIÊN TRONG 1 LỚP

```sql
create function fn_DemSVLop
(
    @MaLop varchar(10)
)
returns int
as
begin
    declare @SoSinhVien int;

    select @SoSinhVien = count(*)
    from SinhVien
    where MaLop = @MaLop;

    return @SoSinhVien;
end;
go

select dbo.fn_DemSVLop('L02') as SoSinhVien;
go
```

- Tầng DTO: Khai báo lớp SinhVienDTO chứa các thuộc tính (MaSV, HoTen, MaLop, Email,...). Biến MaLop làm tham số truyền dữ liệu từ giao diện lên các tầng trên.
- Tầng BUS: Tiếp nhận MaLop, kiểm tra hợp lệ (không để trống), sau đó gọi phương thức DemSinhVienTheoLop(maLop) ở tầng DAL.
- Tầng DAL: Kết nối CSDL, gọi Function qua lệnh SELECT dbo.fn_DemSVLop(@MaLop), sử dụng cmd.ExecuteScalar() để lấy giá trị đơn và trả về kiểu int cho BUS.
- Tầng SQL: Xây dựng Scalar Function fn_DemSVLop nhận tham số @MaLop (varchar), dùng lệnh SELECT COUNT(*) để đếm số sinh viên theo mã lớp và trả về kết quả bằng RETURN.

TRIGGER

Trigger là một đoạn thủ tục SQL được thực thi tự động khi 1 sự kiện cụ thể xảy ra trên 1 bảng (table) hoặc dạng xem (view)

ĐẶC ĐIỂM NỔI BẬT

Chức năng và mục đích chính

Tự động thực thi các tác vụ ngầm:

- Kiểm tra tính hợp lệ của dữ liệu, thực thi quy tắc nghiệp vụ (Business Logic)
- Ngăn chặn các thao tác sai trái, hoặc tự động đồng bộ và cập nhật dữ liệu liên quan.

Thời điểm thực thi

Được chia dựa trên thời điểm chạy so với sự kiện:

- Trigger Before: Thực thi trước khi sự kiện chính diễn ra.
- Trigger After: Thực thi sau khi sự kiện chính đã hoàn tất.

Sự kiện kích hoạt tự động

Không gọi thủ công như Store Procedure hay Function mà tự động chạy khi xảy ra các sự kiện:

- DML Event: Thay đổi dữ liệu (INSERT, UPDATE, DELETE) trên bảng hoặc view.
- DDL Event: Thay đổi cấu trúc cơ sở dữ liệu (CREATE, ALTER, DROP).
- Logon Event (SQL Server): Người dùng đăng nhập vào hệ thống (dùng để kiểm soát hoặc giới hạn quyền truy cập).

TRIGGER

CÚ PHÁP TẠO TRIGGER TRONG SQL

Tạo Trigger

```
CREATE TRIGGER
trigger_name
ON table_name
FOR HOẶC AFTER { INSERT |
UPDATE | DELETE}
AS
BEGIN
        -- CODE THỰC THI
END;
```

TRIGGER

ƯU ĐIỂM VÀ NHƯỢC ĐIỂM

Ưu Điểm

- Bắt lỗi business logic trực tiếp ngay ở mức cơ sở dữ liệu.
- Là giải pháp thay thế hiệu quả cho các công việc cần hẹn giờ theo lịch.
- Rất hiệu quả trong việc theo dõi và kiểm tra các thay đổi dữ liệu trong bảng.

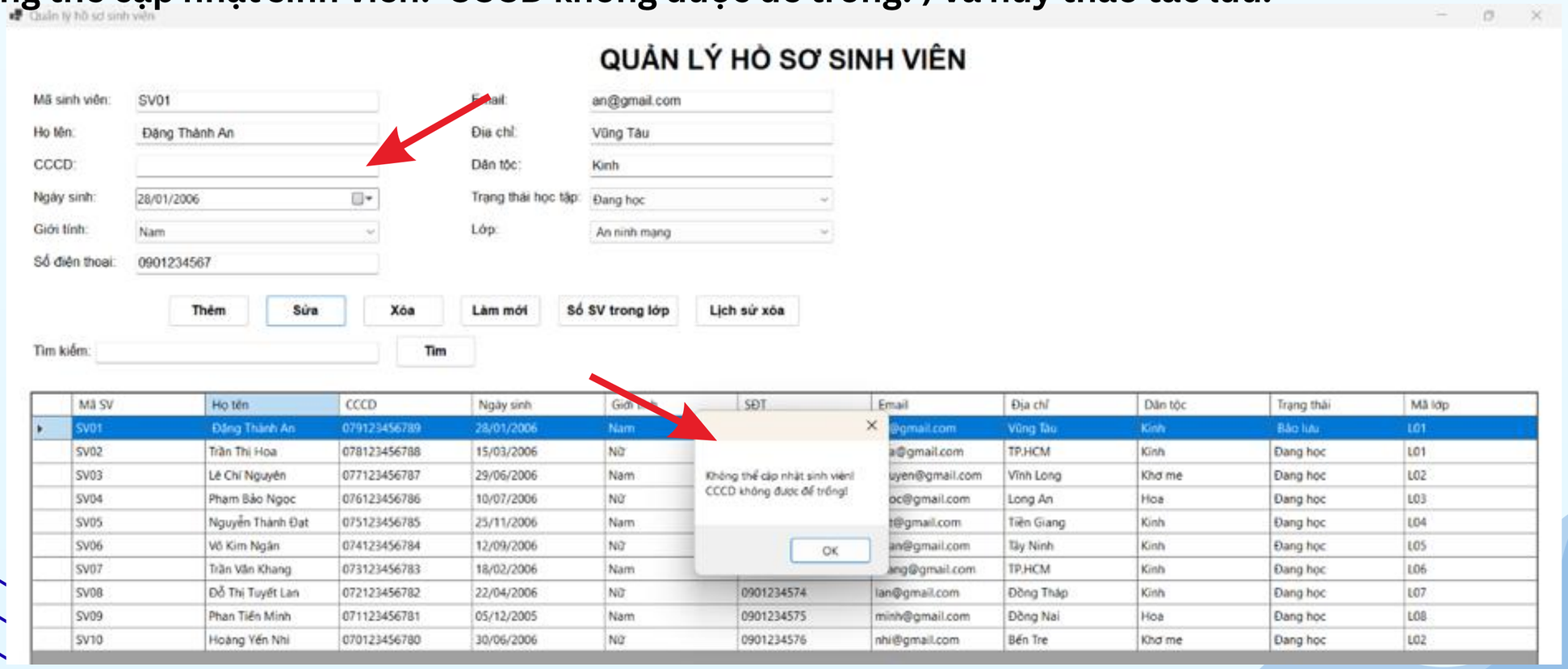
Nhược điểm

- Chỉ đóng vai trò bổ trợ kiểm tra tính hợp lệ, không thay thế hoàn toàn được các biện pháp kiểm tra ở tầng ứng dụng.
- Hoạt động ngầm trong CSDL và không hiển thị ở tầng giao diện, khiến việc gỡ lỗi hoặc xác định nguyên nhân sự cố trở nên khó khăn.
- Làm tăng khối lượng công việc cho CSDL (do tự động thực hiện các câu lệnh update ngầm) và có thể làm giảm hiệu năng hệ thống.

THỰC THI

TẠO TRIGGER KIỂM TRA KHI CẬP NHẬT CCCD CỦA SINH VIÊN

- **Bước 1: Chọn thông tin sinh viên cần cập nhật trên danh sách (ví dụ: sinh viên SV1 - Đặng Thành An) và chỉnh sửa lại trường thông tin CCCD trên form (để trống hoặc nhập khoảng trắng).**
- **Bước 2: Nhấn vào nút Sửa để thực hiện cập nhật thông tin.**
- **Bước 3: Hệ thống hiển thị hộp thoại thông báo lỗi từ Trigger kích hoạt khi cập nhật dữ liệu với cảnh báo (Không thể cập nhật sinh viên! 'CCCD không được để trống!') và hủy thao tác lưu.**



THỰC THI

TẠO TRIGGER KIỂM TRA KHI CẬP NHẬT CCCD CỦA SINH VIÊN

```
1065    -- Tạo trigger kiểm tra khi cập nhật CCCD của sinh viên
1066    create trigger trg_KTCapNhatSV
1067    ON dbo.SinhVien
1068    AFTER UPDATE
1069    AS
1070    BEGIN
1071        SET NOCOUNT ON;
1072
1073        -- Nếu CCCD bị NULL hoặc để trống
1074        IF EXISTS
1075        (
1076            SELECT 1
1077            FROM inserted
1078            WHERE CCCD IS NULL
1079               OR LTRIM(RTRIM(CCCD)) = ''
1080        )
1081        BEGIN
1082            -- Hủy thao tác UPDATE
1083            ROLLBACK TRANSACTION;
1084
1085            -- Trả thông báo lỗi về cho ứng dụng C#
1086            THROW 50001,
1087                  N'CCCD không được để trống!',
1088                  1;
1089        END
1090    END;
1091    GO
1092
```

Nhiệm vụ:

- Xây dựng Trigger trg_KTCapNhatSV trên bảng SinhVien với sự kiện AFTER UPDATE nhằm kiểm tra tính hợp lệ của dữ liệu khi sinh viên được cập nhật thông tin.

Nội dung thực thi:

- Sử dụng bảng ảo inserted để kiểm tra giá trị của bản ghi sau khi cập nhật.
- Kiểm tra điều kiện nếu trường CCCD bị NULL hoặc để trống (LTRIM(RTRIM(CCCD)) = '').
- Nếu vi phạm, trigger thực hiện lệnh ROLLBACK TRANSACTION để hủy bỏ toàn bộ thao tác cập nhật và dùng lệnh THROW 50001 để trả thông báo lỗi về ứng dụng C# với nội dung “CCCD không được để trống!”.

MẤT DỮ LIỆU CẬP NHẬT (LOST UPDATE)

XẢY RA KHI HAI GIAO TÁC CÙNG ĐỌC VÀ CẬP NHẬT MỘT DỮ LIỆU

KẾT QUẢ CẬP NHẬT CỦA GIAO TÁC THỰC HIỆN TRƯỚC BỊ GIAO TÁC THỰC HIỆN SAU GHI ĐÈ LÊN

LÀM CHO CSDL KHÔNG PHẢN ÁNH CHÍNH XÁC CÁC GIAO DỊCH ĐÃ XẢY RA

T1	T2
Đọc số dư 100.000đ	
	Đọc số dư 100.000đ
Tiêu 20.000 trong tài khoản	
	Tiêu 30.000 trong tài khoản

SỐ DƯ T1 CẬP NHẬT = 80.000Đ

SỐ DƯ T2 CẬP NHẬT = 70.000Đ

DỮ LIỆU THỰC TẾ CHỈ CÒN 50.000Đ

=> LOST UPDATE KHI T2 GHI ĐÈ LÊN KẾT QUẢ CỦA T1

TRƯỜNG HỢP XẢY RA LOST UPDATE

```
ELSE
BEGIN
    SELECT @OldSDT = SDT, @OldDiaChi = DiaChi
    FROM dbo.NguoiThan
    WHERE MaNT = 'NT01';
END;
```

T1,T2 CÙNG ĐỌC MỘT DỮ LIỆU CŨ (SĐT VÀ ĐỊA CHỈ BAN ĐẦU)

```
IF @Scenario = 'LU'
BEGIN
    UPDATE dbo.NguoiThan
    SET SDT = '0901234568',
        DiaChi = N'Vũng Tàu'
    WHERE MaNT = 'NT01';
END;
```

DỮ LIỆU BAN ĐẦU KHI CHƯA THỰC HIỆN BẤT KÌ GIAO TÁC NÀO

```
UPDATE dbo.NguoiThan
SET SDT = '0901234111'
WHERE MaNT = 'NT01';

COMMIT TRANSACTION;
```

T2 đã sử dụng dữ liệu cũ được đọc từ trước để cập nhật lại bản ghi sau khi T1 đã COMMIT, khiến giá trị SDT mới của T1 bị ghi đè. Đây chính là hiện tượng Lost Update – mất cập nhật.

T1 UPDATE VÀ THAY ĐỔI SĐT CŨ THÀNH MỚI

```
UPDATE dbo.NguoiThan
SET SDT = @OldSDT,
    DiaChi = N'Bình Dương'
WHERE MaNT = 'NT01';
```

T2 GHI LẠI SDT CŨ → SDT MỚI CỦA T1 BỊ MẤT

->

```
UPDATE dbo.NguoiThan
SET SDT = @OldSDT,
    DiaChi = N'Bình Dương'
WHERE MaNT = 'NT01';

COMMIT TRANSACTION;
```

T2 TIẾP TỤC CẬP NHẬT ĐỊA CHỈ MỚI → COMMIT TRANSACTION

Demo các vấn đề giao tác

DEMO LOST UPDATE

Lost Update • Dirty Read • Non-repeatable Read • Phantom Read

Lost Update | Dirty Read | Non-repeatable | Phantom Read | Mô phỏng lỗi | Khắc phục | Reset

T1 và T2 cùng mở NT01. T1 đổi SDT thành 0901234111; T2 sửa địa chỉ và lưu toàn bộ dữ liệu đang giữ.

TRANSACTION 1 - USER 1

1. T1 mở NT01
2. T1 lưu SDT 0901234111 + COMMIT

Trạng thái: ② T1 lưu SDT mới 0901234111 và COMMIT

```
[04:23:48.412] ① T1 mở NT01, SDT đang thấy = 0901234568
[04:23:50.480] ② T1 lưu SDT mới 0901234111 và COMMIT
```

TRANSACTION 2 - USER 2

1. T2 mở NT01 từ dữ liệu cũ
2. T2 lưu địa chỉ Bình Dương

Trạng thái: ③ T2 lưu địa chỉ Bình Dương + SDT đang giữ = 0901234568

```
[04:23:49.367] ② T2 đang giữ bản dữ liệu có SDT = 0901234568
[04:23:51.337] ③ T2 lưu địa chỉ Bình Dương + SDT đang giữ = 0901234568
```

DATABASE INSPECTOR - DỮ LIỆU THỰC TẾ

MaSV	MaNT	HoTen	MoiQuanHe	SDT	DiaChi
SV01	NT01	Nguyễn Thị Hương	Mẹ	0901234568	Bình Dương

GIẢI THÍCH HIỆN TƯỢNG / CƠ CHẾ

LOST UPDATE - MẤT DỮ LIỆU CẬP NHẬT

• Hiện tượng:
Sinh viên và Chuyên viên nhập liệu cùng mở thông tin người thân NT01 khi SDT hiện tại là 0901234568.

Sinh viên cập nhật SDT thành 0901234111 và lưu thành công. Tuy nhiên Chuyên viên vẫn đang giữ dữ liệu cũ có SDT = 0901234568. Khi Chuyên viên sửa địa chỉ thành Bình Dương và lưu toàn bộ dòng, SDT cũ 0901234568 được ghi trở lại.

→ Kết quả: thay đổi SDT 0901234111 của Sinh viên bị mất.

Chế độ: MÔ PHỎNG LỖI

TRƯỜNG HỢP KHẮC PHỤC LOST UPDATE

```
IF @Mode = 'FIX'
BEGIN
    SELECT @OldSDT = SDT, @OldDiaChi = DiaChi
    FROM dbo.NguoiThan WITH (UPDLOCK, ROWLOCK)
    WHERE MaNT = 'NT01';
END
```

T1 ĐỌC DỮ LIỆU CŨ TỪ MANT ‘NT01’

UPDLOCK → CÓ THỂ UPDATE SAU KHI ĐỌC XONG

ROWLOCK → CHỈ KHÓA DỮ LIỆU Ở HÀNG NT01, TRÁNH PHẠM VI LỚN

```
BEGIN TRANSACTION;

IF @Mode = 'FIX'
BEGIN
    EXEC dbo.sp_Demo_Log @RunId, 'T2', 'WAIT',
        N'① T2 mở NT01 bằng UPDLOCK – sẽ chờ nếu T1 đang giữ khóa';

    SELECT @OldSDT = SDT, @OldDiaChi = DiaChi
    FROM dbo.NguoiThan WITH (UPDLOCK, ROWLOCK)
    WHERE MaNT = 'NT01';
END
```

ĐIỀU NÀY KHIẾN T2 PHẢI CHỜ SAU KHI T1 THỰC HIỆN XONG

T2 DÙNG UPDLOCK ĐỂ TRÁNH KHI T2 ĐANG ĐỌC VÀ CÓ GIAO TÁC CHEN VÀO

```
IF @Scenario = 'LU'
BEGIN
    UPDATE dbo.NguoiThan
    SET SDT = '0901234568',
        DiaChi = N'Vũng Tàu'
    WHERE MaNT = 'NT01';
END;
```

DỮ LIỆU BAN ĐẦU KHI CHƯA THỰC HIỆN BẤT KÌ GIAO TÁC NÀO

SỬ DỤNG UPDLOCK GIÚP TRANSACTION 1 GIỮ KHÓA CẬP NHẬT TRÊN DỮ LIỆU NGAY TỪ BƯỚC ĐỌC

↓

T2 PHẢI CHỜ CHO ĐẾN KHI T1 HOÀN TẤT. NHỜ ĐÓ, T2 ĐỌC ĐƯỢC DỮ LIỆU MỚI NHẤT VÀ KHÔNG GHI ĐÈ LÊN CẬP NHẬT CỦA T1

↓

KHẮC PHỤC HIỆN TƯỢNG LOST UPDATE.

Demo các vấn đề giao tác

KHẮC PHỤC LOST UPDATE

Lost Update • Dirty Read • Non-repeatable Read • Phantom Read

Lost Update | Dirty Read | Non-repeatable | Phantom Read | Mô phỏng lỗi | Khắc phục | Reset

T1 và T2 cùng mở NT01. T1 đổi SDT thành 0901234111; T2 sửa địa chỉ và lưu toàn bộ dữ liệu đang giữ. Chế độ fix dùng UPDLOCK ngay từ bước đọc.

TRANSACTION 1 - USER 1

1. T1 mở NT01 bằng UPDLOCK
2. T1 lưu SDT 0901234111 + COMMIT

Trạng thái: ② T1 lưu SDT mới 0901234111 và COMMIT

```
[04:26:33.746] ① T1 mở NT01, SDT đang thấy = 0901234568
[04:26:35.338] ② T1 lưu SDT mới 0901234111 và COMMIT
```

TRANSACTION 2 - USER 2

1. T2 mở NT01 bằng UPDLOCK
2. T2 lưu địa chỉ Bình Dương

Trạng thái: ③ T2 lưu địa chỉ Bình Dương + SDT đang giữ = 0901234111

```
[04:26:34.384] ① T2 mở NT01 bằng UPDLOCK - sẽ chờ nếu T1 đang giữ khóa
[04:26:35.338] ② T2 đang giữ bản dữ liệu có SDT = 0901234111
[04:26:36.136] ③ T2 lưu địa chỉ Bình Dương + SDT đang giữ = 0901234111
```

DATABASE INSPECTOR - DỮ LIỆU THỰC TẾ

MaSV	MaNT	HoTen	MoiQuanHe	SDT	DiaChi
SV01	NT01	Nguyễn Thị Hương	Mẹ	0901234111	Bình Dương

GIẢI THÍCH HIỆN TƯỢNG / CƠ CHẾ

KHẮC PHỤC LOST UPDATE

• Sử dụng UPDLOCK khi đọc thông tin NT01.

Khi Transaction 1 đã mở NT01 bằng UPDLOCK, Transaction 2 không thể đồng thời lấy khóa cập nhật trên cùng bản ghi mà phải chờ Transaction 1 hoàn tất.

Sau khi T1 COMMIT SDT = 0901234111, T2 mới được đọc dữ liệu mới nhất rồi sửa địa chỉ thành Bình Dương.

→ Kết quả cuối: SDT 0901234111 vẫn được giữ, chỉ DiaChi thay đổi.

Chế độ: KHẮC PHỤC

ĐỌC DỮ LIỆU RÁC - DIRTY READ

XẢY RA KHI MỘT GIAO TÁC T2 ĐỌC DỮ LIỆU MÀ GIAO TÁC T1 ĐÃ THAY ĐỔI NHƯNG CHƯA COMMIT

↓

NẾU GIAO TÁC T1 THỰC HIỆN THAY ĐỔI SAU ĐÓ BỊ ROLLBACK, DỮ LIỆU MÀ GIAO TÁC T2 ĐÃ ĐỌC TRỞ THÀNH DỮ LIỆU KHÔNG HỢP LỆ

↓

LÀM CHO CSDL KHÔNG PHẢN ÁNH CHÍNH XÁC CÁC GIAO DỊCH ĐÃ XẢY RA

T1	T2
Cập nhật số dư từ 10tr thành 5tr đồng	
	Đọc số dư và nhận 5tr đồng
T1 ROLLBACK → TRẢ VỀ 10tr ĐỒNG	

T1 CẬP NHẬT SỐ DƯ 10TR → 5 TR
T2 ĐỌC SỐ DƯ MÀ T1 ĐANG THỰC HIỆN

↘

THỰC TẾ T1 CHƯA HOÀN THÀNH GIAO TÁC VÀ BỊ LỖI PHẢI ROLLBACK VỀ

↙

T2 ĐỌC SAI DỮ LIỆU HAY CÒN GỌI LÀ DỮ LIỆU RÁC KHÔNG CÓ THẬT

↘

XẢY RA DIRTY READ TRONG GIAO TÁC

TRƯỜNG HỢP XẢY RA DIRTY READ

```
IF @Scenario = 'DR'
BEGIN
    UPDATE dbo.SinhVien
    SET TrangThaiHocTap = N'Đang học'
    WHERE MaSV = 'SV01';
END;
```

DỮ LIỆU BAN ĐẦU KHI CHƯA THỰC HIỆN BẤT KÌ GIAO TÁC NÀO

```
UPDATE dbo.SinhVien
SET TrangThaiHocTap = N'Bảo lưu'
WHERE MaSV = 'SV01';

EXEC dbo.sp_Demo_Log @RunId, 'T1', 'DANGER',
    N'② UPDATE SV01: Đang học → Bảo lưu (CHƯA COMMIT)';
```

T1 UPDATE VÀ THAY ĐỔI SV01 → BẢO LƯU (CHƯA COMMIT)

=> THAY ĐỔI NÀY CHƯA TRỞ THÀNH DỮ LIỆU CHÍNH THỨC

```
SET @Msg =
    N'② T2 đọc được TrangThaiHocTap = ' +
    ISNULL(@TrangThai, N'NULL');

EXEC dbo.sp_Demo_Log
    @RunId, 'T2', 'SUCCESS', @Msg;
```

XÁC NHẬN TRẠNG THÁI MÀ T2 ĐỌC ĐƯỢC (NẾU NULL THÌ GHI VÀO NULL)

```
WHILE 1 = 1
BEGIN
    SELECT @Action = T1Action, @Cancelled = Cancelled
    FROM dbo.DemoControl WITH (NOLOCK)
    WHERE RunId = @RunId;

    IF @Cancelled = 1
    BEGIN
        ROLLBACK TRANSACTION;
        RETURN;
    END;

    IF @Action IN ('COMMIT', 'ROLLBACK') BREAK;
    WAITFOR DELAY '00:00:00.150';
END;
```

GIÚP DỪNG T1, TẠO CƠ HỘI CHO T2 THỰC HIỆN VIỆC ĐỌC DỮ LIỆU T1 THAY ĐỔI = READ UNCOMMITTED

```
IF @Mode = 'ERROR'
BEGIN
    EXEC dbo.sp_Demo_Log @RunId, 'T2', 'DANGER',
        N'① T2 dùng READ UNCOMMITTED – có thể đọc dữ liệu chưa COMMIT';

    SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

READ UNCOMMITTED → T2 KHÔNG CẦN CHỜ T1 HOÀN TẤT MÀ CÓ THỂ ĐỌC DỮ LIỆU T1 ĐANG THAY ĐỔI.

TRƯỜNG HỢP XẢY RA DIRTY READ

```
IF @Action = 'COMMIT'

BEGIN
    COMMIT TRANSACTION;
    EXEC dbo.sp_Demo_Log @RunId, 'T1', 'SUCCESS',
        N'③ COMMIT - Bảo lưu trở thành dữ liệu chính thức';
END
ELSE
BEGIN
    ROLLBACK TRANSACTION;
    EXEC dbo.sp_Demo_Log @RunId, 'T1', 'SUCCESS',
        N'③ ROLLBACK - SV01 trở lại Đang học';
END;
```

CÓ HAI TRƯỜNG HỢP XẢY RA:

=> T1 COMMIT → BẢO LƯU THÀNH DỮ LIỆU CHÍNH THỨC → THÌ T2 VẪN ĐỌC LÀ DỮ LIỆU RÁC CHƯA CHÍNH THỨC

=> T1 ROLLBACK → TRẢ SV 01 THÀNH ĐANG HỌC → THÌ T2 ĐÃ ĐỌC DỮ LIỆU RÁC KHÔNG CÓ THẬT

KẾT QUẢ CHO THẤY TRANSACTION 2 ĐÃ ĐỌC ĐƯỢC GIÁ TRỊ “BẢO LƯU” TRONG KHI TRANSACTION 1 CHƯA COMMIT

↓

TẠI THỜI ĐIỂM T2 ĐỌC, ĐÂY VẪN LÀ DỮ LIỆU CHƯA ĐƯỢC XÁC NHẬN

↙ ↘

NẾU T1 THỰC HIỆN ROLLBACK, GIÁ TRỊ “BẢO LƯU” SẼ BỊ HỦY VÀ DỮ LIỆU TRỞ VỀ “ĐANG HỌC”

↓

CHỨNG MINH T2 ĐÃ ĐỌC PHẢI DỮ LIỆU CHƯA COMMIT

NẾU T1 THỰC HIỆN COMMIT, GIÁ TRỊ “BẢO LƯU” SẼ ĐƯỢC TRỞ THÀNH DỮ LIỆU CHÍNH THỨC

↓

CHỨNG MINH T2 ĐÃ ĐỌC PHẢI DỮ LIỆU CHƯA COMMIT DO T2 ĐÃ ĐỌC DỮ LIỆU LÚC T1 CHƯA COMMIT (ĐỢI MÃI TỚI T1 COMMIT THÌ T2 ĐÃ ĐỌC DỮ LIỆU TRƯỚC ĐÓ RỒI)

↘ ↙

=> XẢY RA TRƯỜNG HỢP DIRTY READ

Demo các vấn đề giao tác

DEMO DIRTY READ

Lost Update • Dirty Read • Non-repeatable Read • Phantom Read

Lost Update | Dirty Read | Non-repeatable | Phantom Read | Mô phỏng lỗi | Khắc phục | Reset

SV01: Đang học → Bảo lưu nhưng T1 chưa COMMIT. T2 dùng READ UNCOMMITTED để đọc dữ liệu chưa xác nhận.

TRANSACTION 1 - USER 1

1. Bắt đầu giao tác
2. Cập nhật SV01 → Bảo lưu (CHƯA COMMIT)
3a. ✓ COMMIT
3b. ✗ ROLLBACK

Trạng thái: ③ ROLLBACK - SV01 trở lại Đang học

```
[04:27:20.708] ① BEGIN TRANSACTION - Phòng Đào tạo bắt đầu giao tác
[04:27:21.822] ② UPDATE SV01: Đang học → Bảo lưu (CHƯA COMMIT)
[04:27:25.005] ③ ROLLBACK - SV01 trở lại Đang học
```

TRANSACTION 2 - USER 2

Tra cứu SV01 - READ UNCOMMITTED

Trạng thái: ② T2 đọc được TrangThaiHocTap = Bảo lưu

```
----- KẾT QUẢ STORED PROCEDURE -----
MucCoLap: READ UNCOMMITTED
MaSV: SV01
HoTen: Nguyễn Văn An
TrangThaiHocTap: Bảo lưu
MaLop: L01
------------------------------
[04:27:23.016] ① T2 dùng READ UNCOMMITTED - có thể đọc dữ liệu chưa COMMIT
[04:27:23.032] ② T2 đọc được TrangThaiHocTap = Bảo lưu
```

DATABASE INSPECTOR - DỮ LIỆU THỰC TẾ

MaSV	HoTen	TrangThaiHocTap	MaLop
SV01	Nguyễn Văn An	Đang học	L01

GIẢI THÍCH HIỆN TƯỢNG / CƠ CHẾ

DIRTY READ - ĐỌC DỮ LIỆU RÁC

• Hiện tượng:
Phòng Đào tạo cập nhật trạng thái của SV01 từ "Đang học" thành "Bảo lưu" nhưng Transaction 1 chưa COMMIT.

Trong thời điểm đó, Transaction 2 sử dụng READ UNCOMMITTED và vẫn đọc được trạng thái "Bảo lưu".

Sau đó Transaction 1 ROLLBACK nên dữ liệu thật của SV01 trở lại "Đang học".

› Như vậy giá trị "Bảo lưu" mà T2 đã đọc là dữ liệu chưa được xác nhận, tức dữ liệu rác

Chế độ: MÔ PHỎNG LỖI

TRƯỜNG HỢP KHẮC PHỤC DIRTY READ

```
IF @Scenario = 'DR'
BEGIN
    UPDATE dbo.SinhVien
    SET TrangThaiHocTap = N'Đang học'
    WHERE MaSV = 'SV01';
END;
```

DỮ LIỆU BAN ĐẦU KHI CHƯA THỰC HIỆN BẤT KÌ GIAO TÁC NÀO

```
ELSE
BEGIN
    EXEC dbo.sp_Demo_Log @RunId, 'T2', 'WAIT',
        N'① T2 dùng READ COMMITTED – đang chờ T1 kết thúc nếu còn khóa';

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

    SELECT @TrangThai = TrangThaiHocTap
    FROM dbo.SinhVien
    WHERE MaSV = 'SV01';
END;
```

TRÊN SP ĐÃ HIỂN THỊ T2 ĐANG CHỜ ĐỂ ĐỌC DỮ LIỆU COMMIT TỪ T1

=>READ COMMITTED YÊU CẦU T2 KHÔNG ĐƯỢC ĐỌC DỮ LIỆU CHƯA COMMIT

```
    SELECT @TrangThai = TrangThaiHocTap
    FROM dbo.SinhVien
    WHERE MaSV = 'SV01';
END;

SET @Msg =
    N'② T2 đọc được TrangThaiHocTap = ' +
    ISNULL(@TrangThai, N'NULL');
```

T2 ĐỌC DỮ LIỆU CỦA SV01

NẾU TRẠNG THÁI = NULL THÌ NHẬN NULL

TRANSACTION 2 SỬ DỤNG MỨC CÔ LẬP READ COMMITTED NÊN KHÔNG THỂ ĐỌC DỮ LIỆU CHƯA ĐƯỢC TRANSACTION 1 COMMIT

↓

KHI T1 CHƯA KẾT THÚC GIAO TÁC, T2 PHẢI CHỜ

↓

SAU KHI T1 COMMIT HOẶC ROLLBACK, T2 MỚI ĐỌC DỮ LIỆU CHÍNH THỨC

↓

HIỆN TƯỢNG DIRTY READ ĐƯỢC KHẮC PHỤC

Demo các vấn đề giao tác

KHẮC PHỤC DIRTY READ

Lost Update • Dirty Read • Non-repeatable Read • Phantom Read

Lost Update | Dirty Read | Non-repeatable | Phantom Read | Mô phỏng lỗi | Khắc phục | Reset

SV01: Đang học → Bảo lưu nhưng T1 chưa COMMIT. T2 dùng READ COMMITTED nên phải chờ T1 COMMIT/ROLLBACK.

TRANSACTION 1 - USER 1

1. Bắt đầu giao tác
2. Cập nhật SV01 → Bảo lưu (CHƯA COMMIT)
3a. ✓ COMMIT
3b. ✗ ROLLBACK

Trạng thái: ③ ROLLBACK - SV01 trở lại Đang học

```
[04:29:13.243] ① BEGIN TRANSACTION - Phòng Đào tạo bắt đầu giao tác
[04:29:14.039] ② UPDATE SV01: Đang học → Bảo lưu (CHƯA COMMIT)
[04:29:16.587] ③ ROLLBACK - SV01 trở lại Đang học
```

TRANSACTION 2 - USER 2

Tra cứu SV01 - READ COMMITTED

Trạng thái: ② T2 đọc được TrangThaiHocTap = Đang học

```
----- KẾT QUẢ STORED PROCEDURE -----
MucCoLap: READ COMMITTED
MaSV: SV01
HoTen: Nguyễn Văn An
TrangThaiHocTap: Đang học
MaLop: L01
------------------------------
[04:29:16.587] ② T2 đọc được TrangThaiHocTap = Đang học
```

DATABASE INSPECTOR - DỮ LIỆU THỰC TẾ

MaSV	HoTen	TrangThaiHocTap	MaLop
SV01	Nguyễn Văn An	Đang học	L01

GIẢI THÍCH HIỆN TƯỢNG / CƠ CHẾ

KHẮC PHỤC DIRTY READ

• Sử dụng READ COMMITTED.

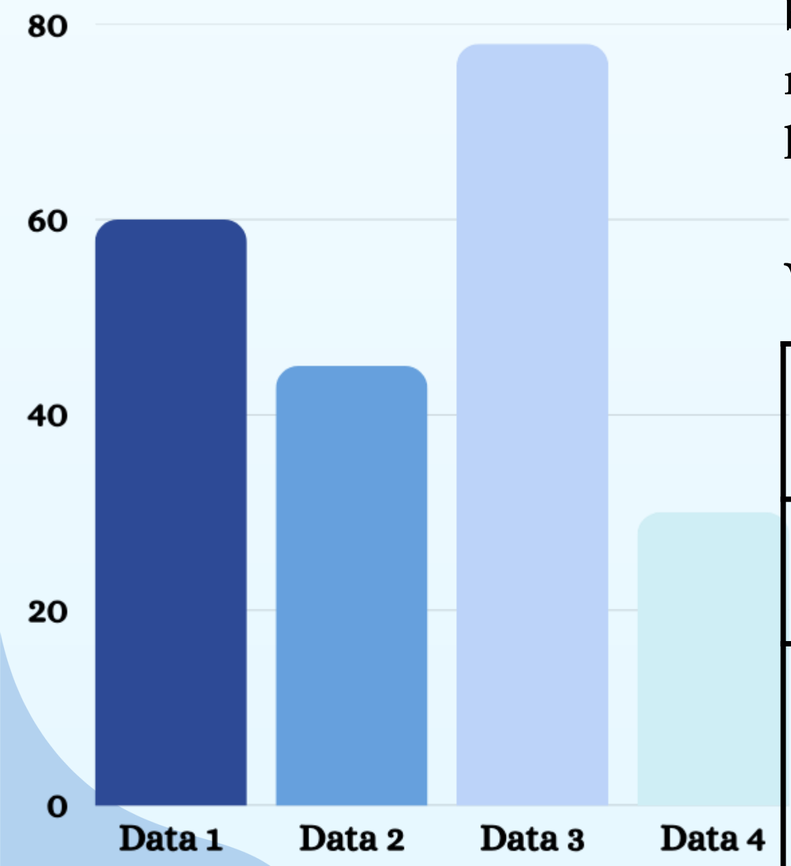
Khi T1 đang thay đổi trạng thái SV01 nhưng chưa COMMIT hoặc ROLLBACK, T2 không được phép đọc giá trị đang thay đổi mà phải chờ T1 kết thúc.

Nếu T1 ROLLBACK, T2 chỉ đọc được trạng thái "Đang học".

→ T2 không còn đọc phải dữ liệu chưa được xác nhận.

Chế độ: KHẮC PHỤC

KHÔNG ĐỌC LẠI ĐƯỢC DỮ LIỆU (NON-REPEATABLE-READ)



Không đọc lại được dữ liệu xảy ra khi một giao tác đọc cùng một bản ghi hai lần, nhưng giữa hai lần đọc, một giao tác khác đã cập nhật và COMMIT bản ghi đó. Vì vậy, cùng một câu lệnh đọc nhưng lại nhận được hai kết quả khác nhau.

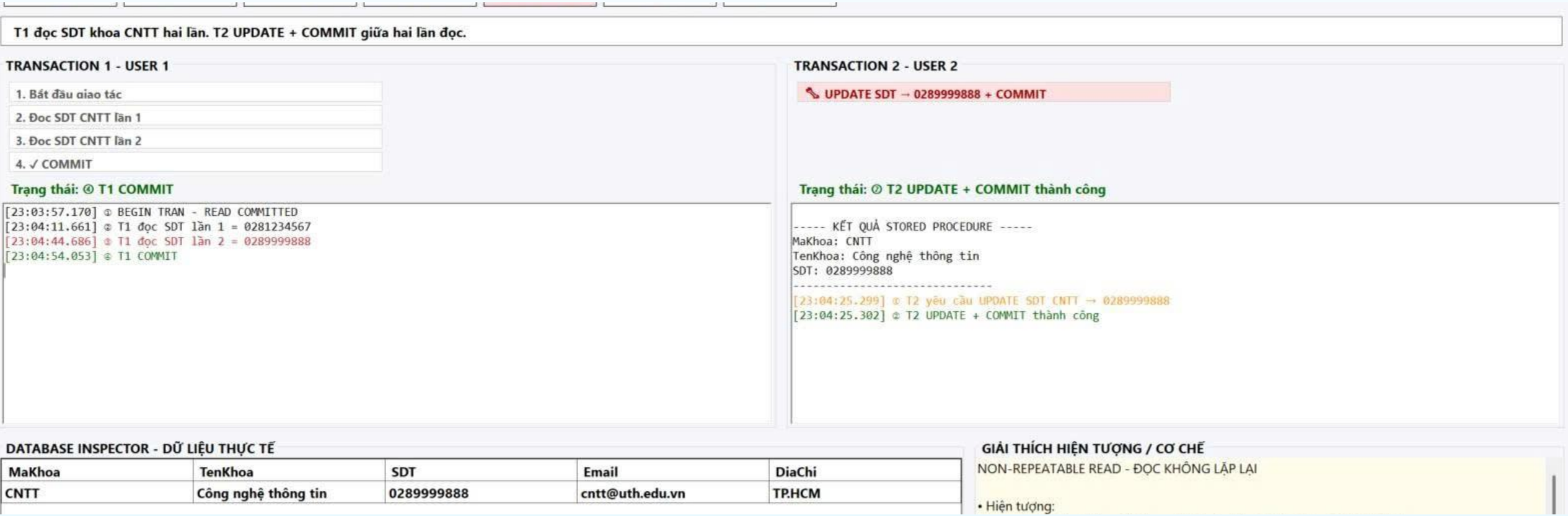
VÍ DỤ

T1	T2
Đọc giá 100.000đ	
	Cập nhật giá thành 120.000đ Commit
Đọc lại giá 120.000đ	

NHƯ VẬY, TRONG CÙNG MỘT GIAO TÁC T1, CÙNG MỘT DỮ LIỆU NHƯNG LẠI NHẬN ĐƯỢC HAI GIÁ TRỊ KHÁC NHAU.

KẾT QUẢ XỬ LÝ CỦA T1 CÓ THỂ KHÔNG NHẤT QUÁN VÌ DỮ LIỆU ĐÃ THAY ĐỔI TRONG KHI GIAO TÁC VẪN ĐANG THỰC HIỆN.

MÔ PHỎNG LỖI – NON-REPEATABLE READ



T1 BẮT ĐẦU GIAO TÁC VỚI MỨC CÔ LẬP READ COMMITTED.

T1 ĐỌC SDT KHOA CNTT LẦN 1: 0281234567

T2 THỰC HIỆN UPDATE SDT → 0289999888 VÀ COMMIT.

T1 ĐỌC LẠI LẦN 2 VÀ NHẬN ĐƯỢC 0289999888.

- HAI LẦN ĐỌC CÙNG MỘT BẢN GHI NHƯNG CHO HAI GIÁ TRỊ KHÁC NHAU → XẢY RA NON-REPEATABLE READ.

CODE SQL GIẢI THÍCH LỖI

```
IF @Mode = 'FIX'
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
ELSE
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

BEGIN TRANSACTION;
```

KHI CHẠY MÔ PHỎNG LỖI T1 SỬ DỤNG READ COMMITTED.

```
SELECT @Lan1 = SDT
FROM dbo.Khoa
WHERE MaKhoa = 'CNTT';
```

T1 ĐỌC LẦN 1 VÀ LƯU CÁC GIÁ TRỊ LẦN 1

```
BEGIN TRANSACTION;

UPDATE dbo.Khoa
SET SDT = '0289999888'
WHERE MaKhoa = 'CNTT';

COMMIT TRANSACTION;
```

T2 THAY ĐỔI DỮ LIỆU VÀ COMMIT, NÊN 0289999888 TRỞ THÀNH DỮ LIỆU CHÍNH THỨC TRONG DATABASE.

CODE SQL GIẢI THÍCH LỖI

```
SELECT @Lan2 = SDT
FROM dbo.Khoa
WHERE MaKhoa = 'CNTT';
```

T1 ĐỌC LẠI LẦN 2 VÀ NHẬN CÁC GIÁ TRỊ MỚI

SAU ĐÓ CODE SO SÁNH:

```
SET @LogType =
    CASE WHEN @Lan1 <> @Lan2
         THEN 'DANGER'
         ELSE 'SUCCESS'
    END;
```

NẾU @LAN1 KHÁC @LAN2 THÌ HỆ THỐNG XÁC ĐỊNH CÓ LỖI NON-REPEATABLE READ.

KHẮC PHỤC LỖI – NON-REPEATABLE READ



T1 BẮT ĐẦU GIAO TÁC VỚI MỨC CÔ LẬP REPEATABLE READ.

T1 ĐỌC SDT KHOA CNTT LẦN 1: 0281234567

T1 ĐỌC SDT LẦN 2 VẪN NHẬN 0281234567.

T1 THỰC HIỆN COMMIT.

T2 YÊU CẦU UPDATE SDT THÀNH 0289999888 NHƯNG PHẢI CHỜ T1 KẾT THÚC.

SAU ĐÓ T2 MỚI UPDATE VÀ COMMIT THÀNH CÔNG.

HAI LẦN ĐỌC CỦA T1 CHO CÙNG MỘT KẾT QUẢ, KHÔNG CÒN NON-REPEATABLE READ.

CODE SQL KHẮC PHỤC LỖI

```
IF @Mode = 'FIX'
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
ELSE
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

BEGIN TRANSACTION;
```

KHI CHẠY KHẮC PHỤC T1 SỬ DỤNG REPEATABLE READ..

```
SELECT @Lan1 = SDT
FROM dbo.Khoa
WHERE MaKhoa = 'CNTT';
```

T1 ĐỌC LẦN 1 VÀ LƯU CÁC GIÁ TRỊ LẦN 1

```
EXEC dbo.sp_Demo_Log @RunId, 'T2', 'WAIT',
    N'① T2 yêu cầu UPDATE SDT CNTT → 0289999888';

BEGIN TRANSACTION;

UPDATE dbo.Khoa
SET SDT = '0289999888'
WHERE MaKhoa = 'CNTT';

COMMIT TRANSACTION;
```

T2 – YÊU CẦU UPDATE CẬP NHẬT SĐT THÀNH 0289999888 NHƯNG PHẢI CHỜ T1 KẾT THÚC.

CODE SQL KHẮC PHỤC LỖI

```
SELECT @Lan2 = SDT
FROM dbo.Khoa
WHERE MaKhoa = 'CNTT';
```

T1 ĐỌC LẠI LẦN 2 GIÁ TRỊ VẪN GIỮA NGUYÊN

```
COMMIT TRANSACTION;
```

T1 COMMIT KẾT THÚC GIAO TÁC VÀ GIẢI PHÓNG KHÓA, SAU ĐÓ T2 MỚI CÓ THỂ UPDATE.

REPEATABLE READ GIỮ SHARED LOCK TRÊN BẢN GHI T1 ĐÃ ĐỌC CHO ĐẾN KHI COMMIT. VÌ VẬY T2 KHÔNG THỂ UPDATE TRONG LÚC T1 CHƯA KẾT THÚC, GIÚP T1 ĐỌC CÙNG MỘT DỮ LIỆU VỚI CÙNG MỘT KẾT QUẢ Ở CẢ HAI LẦN ĐỌC.



HIỆN TƯỢNG BÓNG MA (PHANTOM)

Bóng ma (Phantom) xảy ra khi một giao tác thực hiện cùng một truy vấn nhiều lần nhưng giữa các lần truy vấn, một giao tác khác thêm hoặc xóa các bản ghi thỏa mãn điều kiện truy vấn. Khi truy vấn lại, số lượng bản ghi trả về thay đổi.

VÍ DỤ

GIẢ SỬ T1 THỰC HIỆN TRUY VẤN:

```
SELECT * FROM SINHVIEN WHERE MAKHOA = 'CNTT';
```

·LẦN ĐẦU T1 TRUY VẤN VÀ NHẬN ĐƯỢC 50 SINH VIÊN THUỘC KHOA CNTT

·T2 THÊM MỘT SINH VIÊN MỚI THUỘC KHOA CNTT VÀ COMMIT

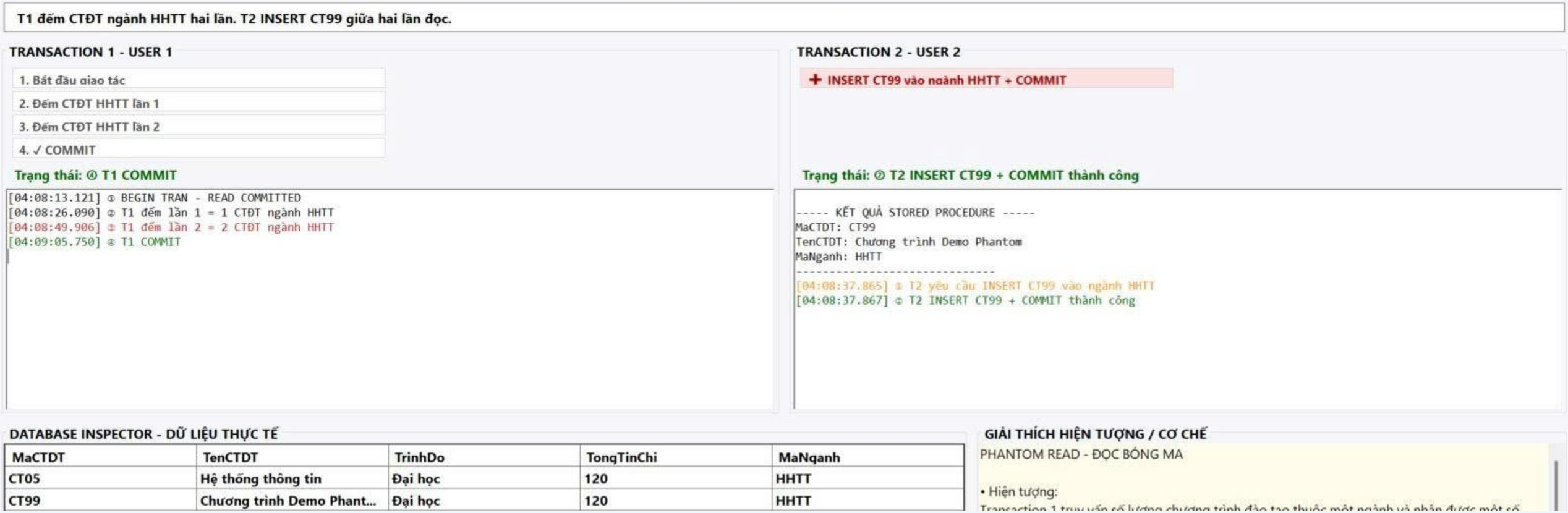
·T1 THỰC HIỆN LẠI CÙNG TRUY VẤN

·LẦN NÀY KẾT QUẢ TRẢ VỀ 51 SINH VIÊN

SINH VIÊN MỚI XUẤT HIỆN TRONG LẦN TRUY VẤN THỨ HAI ĐƯỢC GỌI LÀ BÓNG MA, VÌ BẢN GHI NÀY KHÔNG TỒN TẠI TRONG TẬP KẾT QUẢ CỦA LẦN TRUY VẤN ĐẦU TIÊN.

NHỜ CÁC CƠ CHẾ ĐIỀU KHIỂN TƯƠNG TRANH, HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU CÓ THỂ CHO PHÉP NHIỀU GIAO TÁC THỰC HIỆN ĐỒNG THỜI NHƯNG VẪN DUY TRÌ TÍNH NHẤT QUÁN VÀ ĐỘ TIN CẬY CỦA DỮ LIỆU.

MÔ PHỎNG LỖI – PHANTOM READ



T1 BẮT ĐẦU GIAO TÁC VỚI MỨC CÔ LẬP READ COMMITTED.

T1 ĐỌC SỐ CHƯƠNG TRÌNH ĐÀO TẠO NGÀNH HHTT LẦN 1: 1 CHƯƠNG TRÌNH.

T2 THỰC HIỆN INSERT THÊM MỘT CHƯƠNG TRÌNH MỚI VÀO NGÀNH HHTT VÀ COMMIT.

T1 ĐỌC LẠI LẦN 2: 2 CHƯƠNG TRÌNH.

TRONG CÙNG MỘT GIAO TÁC, T1 ĐỌC CÙNG MỘT ĐIỀU KIỆN NHƯNG SỐ DÒNG THAY ĐỔI DO T2 THÊM DỮ LIỆU.C, T1 ĐỌC CÙNG MỘT ĐIỀU KIỆN NHƯNG SỐ DÒNG THAY ĐỔI DO T2 THÊM DỮ LIỆU.ĐÂY LÀ LỖI PHANTOM READ

CODE SQL GIẢI THÍCH LỖI

```
IF @Mode = 'FIX'
    SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
ELSE
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Ở CHẾ ĐỘ LỖI T1 SỬ DỤNG READ COMMITTED, NÊN T2 CÓ THỂ CHÈN THÊM DÒNG TRONG LÚC T1 ĐANG GIAO TÁC.

```
SELECT @Lan1 = COUNT(*)
FROM dbo.ChuongTrinhDaoTao
WHERE MaNganh = 'HHTT';
```

T1 ĐẾM LẦN 1SỐ CHƯƠNG TRÌNH ĐÀO TẠO THUỘC NGÀNH HHTT. KẾT QUẢ ĐƯỢC LƯU VÀO @LAN1

```
    INSERT INTO dbo.ChuongTrinhDaoTao
    (
        MaCTDT, TenCTDT, TrinhDo,
        ThoiGianDaoTao, TongTinChi,
        NamBatDau, NamKetThuc, MaNganh
    )
    VALUES
    (
        'CT99', N'Chương trình Demo Phantom',
        N'Đại học', 4, 120, 2026, 2030, 'HHTT'
    );
END;

COMMIT TRANSACTION;
```

T2 INSERT THÊM DỮ LIỆUTHÊM CHƯƠNG TRÌNH CT99. CÓ MANGANH = 'HHTT'. SAU ĐÓ T2 COMMIT, NÊN DỮ LIỆU MỚI ĐƯỢC LƯU LẠI.

CODE SQL GIẢI THÍCH LỖI

```
SELECT @Lan2 = COUNT(*)
FROM dbo.ChuongTrinhDaoTao
WHERE MaNganh = 'HHTT';
```

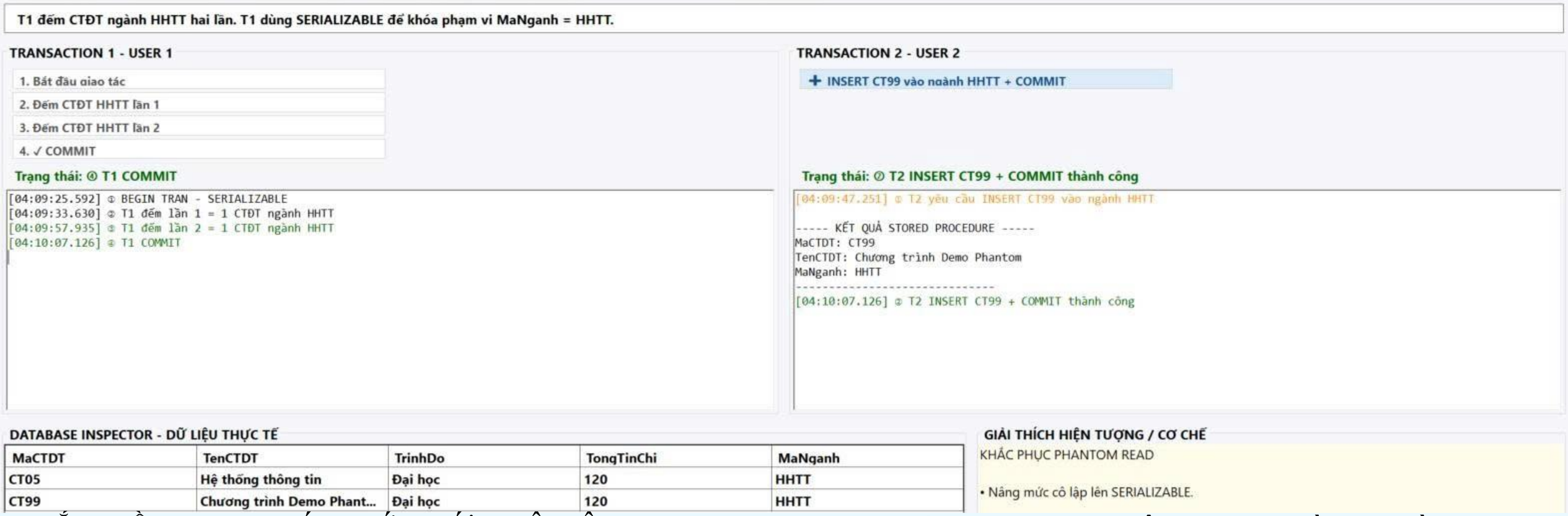
T1 ĐẾM LẠI LẦN 2 LÚC NÀY T2 ĐÃ THÊM CT99, NÊN T1 ĐỌC LẠI:LẦN 2 = 2 CTĐT.

CODE SO SÁNH

```
SET @LogType =
    CASE WHEN @Lan1 <> @Lan2
         THEN 'DANGER'
         ELSE 'SUCCESS'
    END;
```

NẾU @LAN1 KHÁC @LAN2 LÀM SỐ LƯỢNG DÒNG THAY ĐỔI TỪ 1 THÀNH 2. ĐÂY CHÍNH LÀ PHANTOM READ.

KHẮC PHỤC LỖI – PHANTOM READ



T1 BẮT ĐẦU GIAO TÁC VỚI MỨC CÔ LẬP SERIALIZABLE.

T1 ĐẾM SỐ CHƯƠNG TRÌNH NGÀNH HHTT LẦN 1: 1 CHƯƠNG TRÌNH.

T2 THỰC HIỆN INSERT THÊM CT99 VÀO NGÀNH HHTT NHƯNG PHẢI CHỜ, VÌ T1 ĐANG GIỮ KHÓA PHẠM VI.

T1 ĐẾM LẠI LẦN 2: 1 CHƯƠNG TRÌNH.

SAU ĐÓ T2 MỚI THỰC HIỆN INSERT + COMMIT THÀNH CÔNG.

T1

COMMIT.

HAI LẦN ĐỌC CỦA T1 VẪN CHO CÙNG KẾT QUẢ 1 → 1, NÊN PHANTOM READ ĐƯỢC KHẮC PHỤC.

CODE SQL TRONG FILE

CODE SQL KHẮC PHỤC LỖI

```
IF @Mode = 'FIX'
    SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
ELSE
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

BEGIN TRANSACTION;
```

KHI CHẠY CHẾ ĐỘ FIX, T1 SỬ DỤNG SERIALIZABLEMỨC NÀY TẠO RANGE LOCK, GIÚP NGĂN T2 CHÈN THÊM DÒNG VÀO PHẠM VI MÀ T1 ĐANG ĐỌC. .

```
SELECT @Lan1 = COUNT(*)
FROM dbo.ChuongTrinhDaoTao
WHERE MaNganh = 'HHTT';
```

T1 ĐỌC LẦN 1 ĐẾM SỐ CHƯƠNG TRÌNH THUỘC NGÀNH HHTT. ĐƯỢC LƯU VÀO @LAN1

```
        INSERT INTO dbo.ChuongTrinhDaoTao
        (
            MaCTDT, TenCTDT, TrinhDo,
            ThoiGianDaoTao, TongTinChi,
            NamBatDau, NamKetThuc, MaNganh
        )
        VALUES
        (
            'CT99', N'Chương trình Demo Phantom',
            N'Đại học', 4, 120, 2026, 2030, 'HHTT'
        );
    END;

    COMMIT TRANSACTION;
```

T2 YÊU CẦU INSERT MUỐN THÊM CT99 VÀO HHTT NHƯNG DO T1 ĐANG DÙNG SERIALIZABLE, THAO TÁC INSERT PHẢI CHỜ T1 COMMIT.

CODE SQL KHẮC PHỤC LỖI

```
SELECT @Lan2 = COUNT(*)
FROM dbo.ChuongTrinhDaoTao
WHERE MaNganh = 'HHTT';
```

T1 ĐỌC LẦN 2 VÌ CHƯA THỂ INSERT NÊN GIÁ TRỊ 2 LẦN ĐỌC BẰNG NHAU

CODE SO SÁNH

```
SET @LogType =
    CASE WHEN @Lan1 <> @Lan2
         THEN 'DANGER'
         ELSE 'SUCCESS'
    END;
```

SO SÁNH KẾT QUẢ LẦN ĐỌC 1 VÀ LẦN ĐỌC 2 CỦA T1:

NẾU KẾT QUẢ KHÁC NHAU → DANGER → PHÁT HIỆN PHANTOM READ.

NẾU KẾT QUẢ GIỐNG NHAU → SUCCESS → PHANTOM READ ĐƯỢC KHẮC PHỤC.

DEADLOCK



Deadlock xảy ra khi các giao tác chờ khóa của nhau, khiến không giao tác nào có thể tiếp tục thực hiện và hệ thống rơi vào trạng thái tắc nghẽn.

Ví dụ:

- T1 đang giữ khóa Y
- T2 đang giữ khóa X
- T1 muốn khóa X nhưng X đang do T2 giữ
- T2 muốn khóa Y nhưng Y đang do T1 giữ

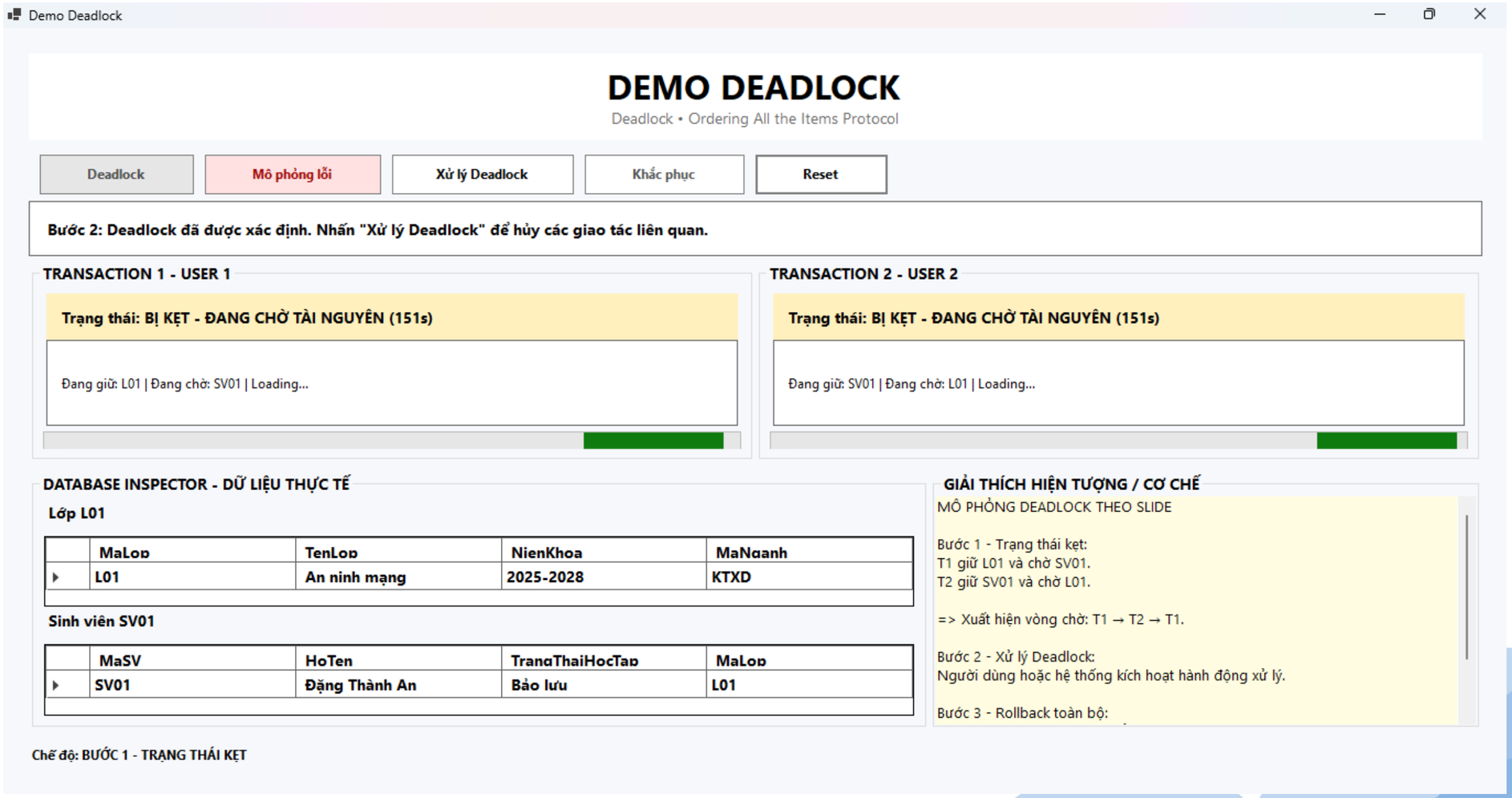
Hai giao tác chờ nhau và không giao tác nào có thể tiếp tục → Deadlock.

Một số giải pháp xử lý Deadlock

- Yêu cầu giao tác xin tất cả các khóa cùng lúc
- Cấp khóa theo một thứ tự nhất định
- Phát hiện Deadlock và thực hiện phục hồi

DEADLOCK

Mô phỏng lỗi



DEADLOCK

Có thể minh họa ngắn:

Transaction 1

↓

Khóa A

↓

Chờ B

↑

Transaction 2

↓

Khóa B

↓

Chờ A

→ T1 chờ T2 và T2 chờ T1 → Deadlock

***4 điều kiện Coffman:** Tình huống mô phỏng thỏa các điều kiện cần thiết để xảy ra Deadlock gồm: độc quyền tài nguyên, giữ và chờ, không thể cưỡng chế tài nguyên và chờ vòng.

DEADLOCK

Code mô phỏng

```
54    -- T1. KHÓA LỚP
55    ----------------
56    begin transaction;
57
58    -- Khóa Lớp 'L01'
59    update Lop
60    set TenLop = N' An ninh mạng'
61    where MaLop = 'L01';
62
63    -- Giữ khóa 10 giây
64    waitfor delay '00:00:10';
65
66    -- Sau đó mới khóa SinhVien 'SV01' để cập nhật trạng thái
67    update SinhVien
68    set TrangThaiHocTap = N' Bảo lưu'
69    where MaSV = 'SV01';
70
71    commit transaction;
72
```

- Transaction 1 bắt đầu giao tác và khóa bản ghi lớp L01 thông qua câu lệnh UPDATE.
- Sau đó sử dụng WAITFOR DELAY để giữ khóa trong một khoảng thời gian trước khi tiếp tục yêu cầu khóa sinh viên SV01.
- Trong thời gian này, T1 giữ tài nguyên L01 và chờ tài nguyên SV01.

DEADLOCK

Code mô phỏng

```
28  ----------------------
29  -- T2. KHÓA SINH VIÊN
30  ----------------------
31  begin transaction;
32
33  -- Khóa Sinh Viên 'SV01'
34  update SinhVien
35  set HoTen = N' Đặng Thành An'
36  where MaSV = 'SV01';
37
38  -- Giữ khóa 10 giây
39  waitfor delay '00:00:10';
40
41  -- Sau đó T2 khóa 'L01' để điều chỉnh niên khóa
42  update Lop
43  set NienKhoa = N'2025-2028'
44  where MaLop = 'L01';
45
46  commit transaction;
47
```

- Transaction 2 bắt đầu giao tác và khóa bản ghi sinh viên SV01 trước.
- Sau đó sử dụng WAITFOR DELAY để giữ khóa trước khi tiếp tục yêu cầu khóa lớp L01.
- Do T1 đang giữ L01 và T2 đang giữ SV01, hai transaction chờ tài nguyên của nhau, tạo thành vòng chờ và dẫn đến Deadlock.

DEADLOCK

Code khắc phục

```
-- NGĂN NGỪA DEADLOCK THEO GIAO THỨC
-- Ordering All the Items Protocol
----------------------------------------

---------------
-- T1. KHÓA LỚP
---------------
begin transaction;

-- Khóa Lớp 'L01'
update Lop
set TenLop = N' An ninh mạng'
where MaLop = 'L01';

-- Giữ khóa 10 giây
waitfor delay '00:00:10';

-- Sau đó mới khóa SinhVien 'SV01' để cập nhật trạng thái
update SinhVien
set TrangThaiHocTap = N' Bảo lưu'
where MaSV = 'SV01';

commit transaction;

```

```

----------------------
-- T2. KHÓA SINH VIÊN
----------------------
begin transaction;

-- T2 cũng phải khóa Lớp 'L01' trước
update Lop
set NienKhoa = N'2025-2028'
where MaLop = 'L01';

-- Giữ khóa 10 giây
waitfor delay '00:00:10';

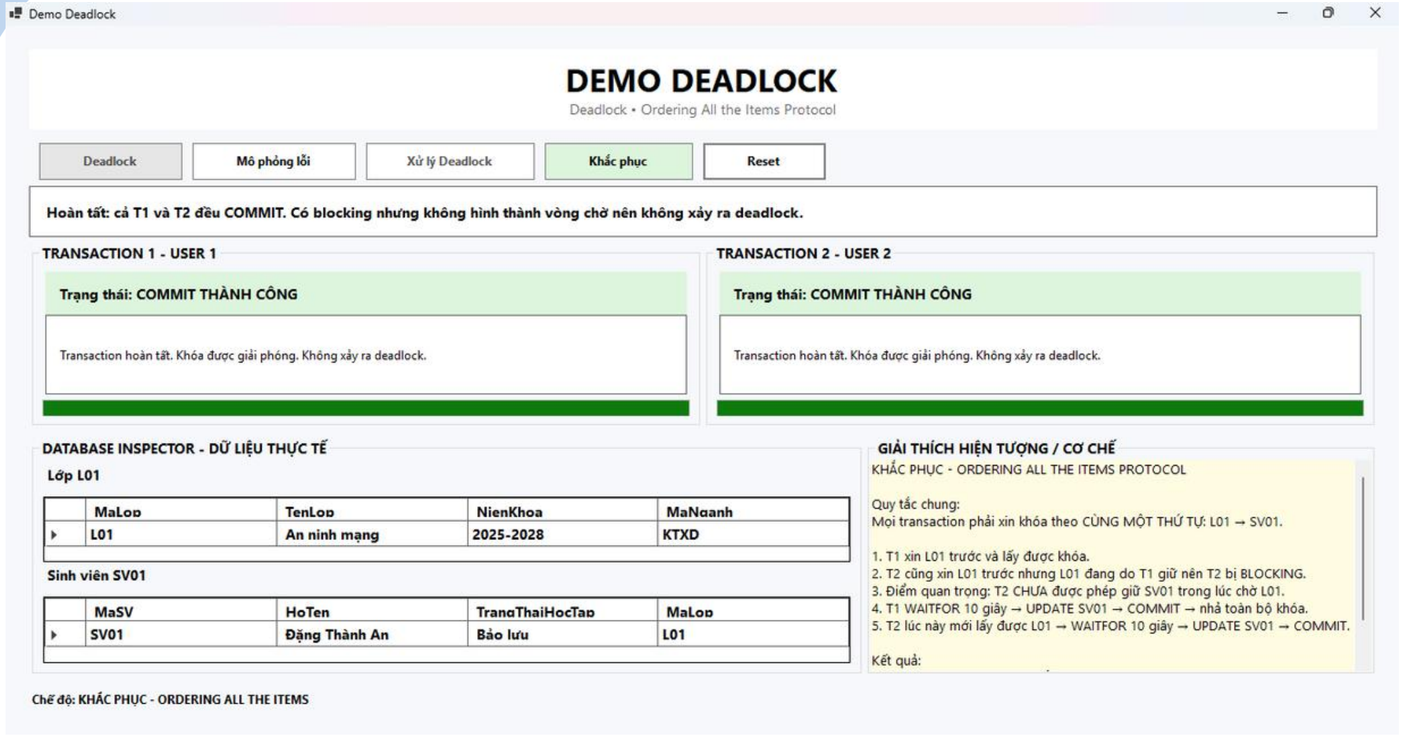
-- Sau đó mới khóa SinhVien 'SV01' để điều chỉnh họ tên
update SinhVien
set HoTen = N' Đặng Thành An'
where MaSV = 'SV01';

commit transaction;

```

DEADLOCK

Khắc phục



- Để ngăn ngừa Deadlock, hệ thống áp dụng giao thức Ordering All the Items, trong đó tất cả các transaction phải truy cập tài nguyên theo cùng một thứ tự.
- Cả T1 và T2 đều khóa L01 trước, sau đó mới thực hiện thao tác trên SV01.
- Vì không còn trường hợp T1 giữ L01 chờ SV01 trong khi T2 giữ SV01 chờ L01 nên vòng chờ được loại bỏ và Deadlock không xảy ra.

THANKS!